

Laconic Cryptography with Preprocessing

Rishabh Bhadauria³ *, Nico Döttling¹, Carmit Hazay², and Chuanwei Lin¹

¹ CISA Helmholtz Center for Information Security
nico.doettling@gmail.com, chuanwei.lin@cisa.de

² Bar-Ilan University
carmit.hazay@biu.ac.il

³ rishabh.bhadauria@gmail.com

Abstract. Laconic cryptography focuses on designing two-message protocols that allow secure computation on large datasets while minimizing communication costs. While laconic cryptography protocols achieve asymptotically optimal communication complexity for many tasks, their concrete efficiency is prohibitively expensive due to the heavy use of public-key techniques or the non-black-box of cryptographic primitives. In this work, we initiate the study of *laconic cryptography with preprocessing*, introducing a model that includes an offline phase to generate database-dependent correlations, which are then used in a lightweight on-line phase. These correlations are conceptually simple, relying on linear-algebraic techniques. This enables us to develop a protocol for private laconic vector oblivious linear evaluation (plvOLE). In such a protocol, the receiver holds a large database DB , and the sender has two messages v and w along with an index i . The receiver learns the value $v \cdot \text{DB}_i + w$ without revealing other information.

Our protocol, which draws from ideas developed in the context of private information retrieval with preprocessing, serves as the backbone for two applications of interest: laconic private set intersection (IPSI) for large universes and laconic function evaluation for RAM-programs (RAM-LFE). Based on our plvOLE protocol, we provide efficient instantiations of these two primitives in the preprocessing model.

1 Introduction

Secure two-party computation (2PC) protocols enable two parties to jointly compute a function over their inputs while revealing no more information than what follows from the output. This powerful tool can capture virtually any cryptographic task. Early results in 2PC established its feasibility [37, 46, 54], but the resulting protocols were prohibitively expensive in several respects. In particular, they introduced significant overheads in round complexity, communication costs, and computational complexity compared to their insecure counterparts. In addition, these protocols were designed for settings in which the computational resources of the participating parties were relatively balanced.

* Work done while at Georgetown University.

To address these bottlenecks, the *offline/online paradigm* introduces a preprocessing phase that generates correlated randomness, shifting expensive computations to an offline phase and reducing the burden during the online phase. Originating from Beaver’s work [4], extensive research has explored various two-party correlations e.g., [5, 7, 9, 10, 13, 18, 22–24, 40, 44, 49, 53]. This paradigm significantly enhances efficiency and simplifies protocols: once preprocessing is complete, the online phase typically involves only lightweight computations, often relying on symmetric-key operations or even information-theoretic techniques. However, it does not shift computational resources between parties to alleviate the burden on constrained participants.

A key primitive in 2PC with preprocessing is *Oblivious Linear Evaluation* (OLE) [19, 39, 48], a generalization of oblivious transfer [51]. In OLE, the receiver holds a field element $x \in \mathbb{F}$ while the sender holds two field elements $v, w \in \mathbb{F}$. At the end of the protocol, the receiver learns the linear function $v \cdot x + w$ and nothing else, while the sender remains oblivious to x . The significance of OLE lies in its *completeness* for secure two-party arithmetic computation [45], meaning that any efficient two-party function can be securely computed given access to a secure OLE protocol. In this work, we will focus on this primitive. As such, it has been extensively studied both theoretically and practically, with a primary focus on reducing the communication complexity of OLE protocols and their associated correlations; see for a few examples [8, 11, 12, 16, 35]. In vector OLE (VOLE), the sender holds two vectors $\mathbf{v}, \mathbf{w} \in \mathbb{F}^k$ such that the receiver learns $\mathbf{v} \cdot x + \mathbf{w}$. VOLE extends the functionality of standard OLE by enabling batch processing of multiple OLE instances in parallel. This makes it a powerful primitive in cryptographic protocols, allowing the efficient generation of multiple OLE correlations from a smaller set of VOLE correlations [2, 8]. As a result, VOLE significantly improves the efficiency of secure computation tasks that rely on OLE.

The advent of fully homomorphic encryption (FHE) [14, 15, 32, 34] had introduced a new approach to secure two-party computation, enabling protocols in just two messages between a receiver and a sender, and with asymptotically optimal communication cost, that grows with the receiver’s input length and the computation’s output. In particular, FHE allows the receiver to delegate almost the entire computational burden to the sender. On the downside, FHE is unsuitable for scenarios where the sender, often a resource-constrained client, cannot handle such a computational load or communication overhead.

An emerging orthogonal trend, termed *laconic cryptography*, addresses the delegation imbalance described above. It focuses on two-message protocols where the receiver has a large input x , while the sender has a small input y . The key requirement for *laconism* is that the communication size be independent of, or at least sublinear in, the size of x . In this setting, the computational burden shifts to the receiver rather than the sender, a concept known as *reverse delegation*. In this context, laconic VOLE enables the receiver to succinctly commit to a large database $\text{DB} \in \{0, 1\}^N$ using a compact hash value. Given this hash, the sender, holding inputs v, w and an index $i \in [N]$, computes a ciphertext c from which

the receiver can decrypt $\mathbf{v} \cdot \text{DB}_i + \mathbf{w}$. Moreover, sender security ensures that the receiver learns nothing beyond this value while communication efficiency is maintained since the hash size remains independent of the database size. As demonstrated below, similarly to how VOLE is fundamental for 2PC, laconic VOLE serves as a crucial building block for laconic protocols.

Laconic VOLE can be seen as an instance of the more general notion of laconic function evaluation (LFE), introduced by Quach et al. [50], which enables the inverse delegation of any function to the receiver. In an LFE protocol, the receiver computes a succinct hash value of a large circuit C . Given this hash value, the sender can encrypt its input y into a ciphertext c , which the receiver can decrypt c to obtain $C(y)$. Security ensures that the receiver learns nothing about y beyond $C(y)$. When two parties wish to compute a function $F(x, y)$ in this setting, the receiver hashes a circuit $C[x](\cdot)$ that evaluates $F(x, \cdot)$ with its input x hardwired. Regarding assumptions and communication efficiency, Quach et al. [50] constructed an LFE scheme for circuits based on the learning with error (LWE) assumption [52] with super-polynomial modulus-to-noise ratio, where the communication scales with the circuit depth but remains independent of its overall size.

The critical feature of laconic protocols is that the receiver can be offline after sending the hash of its large input. The remaining protocol only consists of a single succinct message from the sender to the receiver. In a certain sense, hashing its input C and sending the hash to the sender can be seen as a *non-interactive preprocessing step*.

Thus, it is worth studying the relationship between preprocessing and laconism in MPC in more detail. In a certain way, a preprocessing phase can be seen as a generalization of the hashing step in laconic protocols, with the key distinction that preprocessing protocols are interactive and input-independent. As discussed above, laconic protocols naturally imply an “offline phase” where the receiver sends a laconic, input-dependent message, which is followed by a single-message online phase. In this work, we explore a natural generalization that captures aspects of both the preprocessing paradigm and the setting of laconic cryptography.

Viewing the landscape of laconic protocols, early works focused on laconic OT and related primitives, achieving near-optimal asymptotic communication and computational overheads. However, these constructions relied heavily on non-black-box cryptographic techniques, often embedding costly public-key operations within garbled circuits. As a result, they were primarily feasibility results with limited practical efficiency. More recent works have advanced truly efficient laconic cryptography by avoiding garbled circuits. Among these, Dötting et al. [27] proposed a construction from the LWE assumption, Green et al. [38] introduced one based on a novel assumption over pairing-friendly elliptic curves in the generic group model, and Fleischhacker et al. [29] presented a scheme in the algebraic group model. However, these constructions still rely on non-black-box techniques, limiting their efficiency.

To fully unlock the benefits of laconic protocols in advanced applications, we seek lightweight constructions that minimize the reliance on public-key operations. From a theoretical standpoint, exploring constructions based on weaker assumptions could lead to more efficient and versatile protocols. Toward achieving these goals, we extend the classic laconic setting, introducing an additional communication phase, capturing preprocessing, and studying laconic OLE in this setting.

1.1 Our Results

Inspired by the extensive research on VOLE correlations, we initiate the study of *laconic OLE with preprocessing* and propose a model of *laconic cryptography with preprocessing*. Our contributions focus on constructing *private laconic OLE with preprocessing*, drawing remote inspiration from the recent advances in private information retrieval (PIR) with preprocessing [21]. We present the following key results:

Our Model. We propose a relaxed notion of *laconism* by modeling laconic VOLE with preprocessing in a two-server offline/online paradigm. In our model, two non-colluding servers hold replicas of the database. During the offline phase, the offline server generates *database-dependent correlations* and becomes silent after the preprocessing. In the online phase, the client, acting as the VOLE sender, sends a single message to the online server, which acts as the VOLE receiver. Regarding *laconism*, we ensure that the size of the correlations is sub-linear in the size of the database. Furthermore, the entire protocol consists of only two messages: one from the offline server to the client in the offline phase and another from the client to the online server in the online phase. The key advantage of our new model is that it allows us to decouple the expensive public-key operations from simpler computations by abstracting the particular randomness correlations required for the underlying protocol, shifting the costly computation to the offline phase, and leaving the online phase to handle only lightweight primitives.

Our model can be reduced to the standard two-party setting by having the sender and receiver emulate the role of the offline server, e.g., using laconic function evaluation [50], while maintaining sublinear communication in the overall protocol. Notably, due to the modular nature of our model, any generic or custom protocol can replace the offline server in a black-box manner. Given that the primary goal of this work is to identify the structured correlations that are essential for laconic VOLE, we leave the challenge of developing simplified, customized protocols that can effectively replace the offline server for future study.

Private Laconic Vector-OLE with Preprocessing. Our main contribution is a construction of *private laconic vector OLE* (plvOLE) in the preprocessing model. Recall that in such a protocol the sender’s input consists of a database-index i and two vectors $\mathbf{v}, \mathbf{w}$, and the receiver learns $\mathbf{y} = \mathbf{w} + \mathbf{v} \cdot \text{DB}[i]$.

While the conventional notion of laconic OT [20] leaks the index i to the receiver, our protocol enjoys index privacy, i.e. the receiver learns nothing about i , $\mathbf{v}$ and $\mathbf{w}$ beyond y .

In our protocol, the preprocessing comes in the form of *sparse linear correlations*. Like standard OLE correlations, these laconic OLE correlations are single-use. The use of these correlations allows for the construction of a private laconic OLE protocol with database-dependent preprocessing based solely on symmetric cryptographic primitives or even with information-theoretic security. In our protocol, the offline server generates these correlations locally and subsequently transmits them to the sender for use during the online phase.

Our scheme has the following efficiency parameters, λ is the security parameter and N the database size. We subsume all other parameters such as the size of the underlying field $\mathbb{F}$ in the $\text{poly}(\lambda)$ terms below.

- The communication in the offline phase is $\sqrt{N} \cdot \text{poly}(\lambda)$
- The size of the preprocessed correlations is $\sqrt{N} \cdot \text{poly}(\lambda)$
- The communication in the online phase is $\sqrt{N} \cdot \text{poly}(\lambda)$
- Computational overhead for the sender during the online phase is $\sqrt{N} \cdot \text{poly}(\lambda)$
- Computational overhead for the receiver during the online phase is $\sqrt{N} \cdot \text{poly}(\lambda)$

Our construction is essentially *fully algebraic*, all critical operations just use linear algebra over finite fields. We only use symmetric key cryptography, specifically pseudorandom functions, to locally compress the preprocessed correlations.

Applications. We demonstrate the usefulness of plvOLE by realizing two highly sought-after applications from this notion: private set intersection (PSI) and laconic function evaluation for RAM programs (RAM-LFE). Naturally, both these construction are also in the preprocessing model. Both protocols offer a more efficient alternative to standard protocols, especially in scenarios that align with the two-server model. Specifically, by incorporating preprocessing, query latency is reduced by shifting the heavier computations to the offline phase, making it an ideal solution for time-sensitive operations.

Laconic PSI. Private Set Intersection (PSI) is a cryptographic protocol that enables two parties, each holding a private set of elements, to compute the intersection of their sets without revealing any additional information about their inputs. Due to its broad applicability, including privacy-preserving contact discovery, genomic testing, and secure data analytics, PSI has been widely studied. With increasing input sizes, much of the research has focused on improving communication efficiency, often measured by the ratio of input size to protocol communication complexity. Several paradigms have been proposed to solve PSI, almost all of which support a communication complexity that grows with the length of both input sets.

In practice, however, scenarios often arise where one party’s set is significantly smaller than the other’s. For example, a large database stored on a server may

need to communicate with a resource-constrained client with a much smaller set. In such cases, it is desirable for the client’s workload to be proportional to the size of its set, minimizing computational and communication overhead. Laconic PSI [1, 3] is specifically designed for such settings where the goal is to minimize the communication and computational burden on the party with the smaller set. In Laconic PSI, the server sends a succinct message whose size is independent of its set size, while the client’s computation scales with its input size. This framework is particularly useful in scenarios where a resource-constrained client cannot stay online for prolonged interaction.

Our conceptually simple and efficient laconic PSI protocol is based on our core plvOLE protocol and hence enjoys the same efficiency parameters. It immediately supports large universes $\mathcal{U}$ of exponential size. Moreover, for a receiver set of size N , all parameters are dominated by $\text{poly}(\lambda) \cdot \sqrt{N}$.

RAM-LFE. We further study the notion of laconic function evaluation for RAM programs (RAM-LFE) in the preprocessing model. In the standard setting, the receiver commits to a database DB via a succinct hash value. In our setting, the sender receives plvOLE correlations from the offline server which commit to the database. The sender, whose input is a RAM program P of runtime T , encrypts the program P into a ciphertext c . With the database DB in hand, the receiver can decrypt c to P^{DB} , i.e., the output of P when executed on DB. Regarding security, the receiver must learn nothing about P beyond its output P^{DB} and the runtime T .

For this construction we rely on plvOLE over the field $\mathbb{F}_2$, which corresponds to the notion of private laconic OT. Our construction additionally relies on garbled circuits and achieves the following efficiency parameters, where N is the database size and T the runtime of the RAM-program: Offline communication, online communication and computational overhead for sender and receiver are all $\text{poly}(\lambda) \cdot T \cdot \sqrt{N}$.

Notably, in our construction the garbled circuit do not need to perform any cryptographic operations, but just simple linear algebra over $\mathbb{F}_2$. This is in stark contrast to constructions in the standard laconic cryptography setting, which rely on heavy use of non-black-box techniques [25, 26], and are thus well outside the realm of practically implementable schemes.

1.2 On the Necessity of Database-Dependent Preprocessing

A natural question that arises when considering database-dependent preprocessing is whether this is necessary, or if a database-independent preprocessing could yield the same results. In Section 3 we show that a database-dependent is in fact necessary if we want to solely rely on symmetric-key techniques, as laconic vOLE/OT in a database-independent preprocessing model immediately implies oblivious transfer in the standard model.

1.3 Technical Outline

We will now give an overview of our construction.

Doubly Efficient PIR in the Preprocessing Model Our starting point is the doubly efficient two-server private information retrieval (PIR) protocol of Corrigan-Gibbs and Kogan [21]. They consider the following model:

- In an *offline phase*, a PIR client interacts with an offline server, who is in possession of a database DB of size N to produce a database-dependent *preprocessing cor*. This preprocessing is then locally stored by the client for future use. Two aspects are critical here: First, during the offline phase the client does not yet have an input, hence the preprocessing *cor* is independent of the clients (future) input. Second, both the amount of communication and the size of the preprocessing *cor* are sublinear in the database-size N . In particular, this means that by itself, it does not contain enough information to answer most database queries.
- In the *online phase*, the client gets as input an index $x \in [N]$ and interacts with an online server, who also has access to the database DB. During this interaction, the client *consumes* the preprocessing *cor* but obtains the database entry $\text{DB}[x]$. Two aspects are important here as well: First, the online server is not allowed to communicate with the offline server. Second, we also require that the size of the communication in the online phase is sublinear in the database size N .

In terms of security, we make the usual requirement that the online server learns nothing about the index x . Furthermore, a protocol is called *doubly efficient* in this model if the runtimes of online server and the client in both phases are sublinear in the database size N . The requirement that offline and online servers are not allowed to talk to one another is either enforced by assumption or cryptographically, in the sense that the offline server is only virtual by, e.g., emulating it in a secure two-party computation. The advantage of this model is that it allows for very efficient protocols that can be realized with information-theoretical security or relying on efficient symmetric-key primitives rather than strong public-key primitives, which are inherent in standard PIR [28].

The CK20-Protocol Corrigan-Gibbs and Kogan [21] construct a very elegant protocol that realizes doubly efficient private information retrieval in this model. The main ideas of their protocol can be outlined as follows.

In the offline phase, the client generates $m \approx \lambda\sqrt{N}$ *correlated sets* $S_1, \dots, S_m \subseteq [N]$, and each of these sets is of size $\approx \sqrt{N}$. These sets are generated in such a way that they cover the entire set of indices $[N]$, i.e., it holds that $\bigcup_{i \in [m]} S_i = [N]$. Furthermore, sets $S_1, \dots, S_m$ are generated so that they can be succinctly represented. Specifically, representing all sets jointly requires only a small amount of additional information compared to just representing a single set. In [21], these sets are generated as follows. First, a uniformly random set $S \subseteq [N]$ of size $\sqrt{N}$ is chosen. Additionally, m random *shifts* $\Delta_1, \dots, \Delta_m \in [N]$ are chosen. Now, for each $i \in [m]$, the set S_i is defined as the set of all elements $x \in [N]$ for which $(x - \Delta_i) \bmod N \in S$, i.e. S_i is just a cyclic shift of S by Δ_i . A simple argument shows that the union of the S_i indeed covers $[N]$, except with negligible probability over the choice of the Δ_i . For details, see Section 2.4. Furthermore,

as S is chosen uniformly random, this construction has the feature that each set S_i individually is distributed uniformly random. Note that the joint representation of $S_1, \dots, S_m$ is indeed compact, as S and $\Delta_1, \dots, \Delta_m$ can be represented using $\text{poly}(\lambda) \cdot \sqrt{N}$ bits. Now, in the offline phase the client sends S and $\Delta_1, \dots, \Delta_m$ to the offline server, who computes the parities $h_i = \sum_{j \in S_i} \text{DB}[j]$ for $i = 1, \dots, m$. Here, we assume that the arithmetic is modulo 2. The offline server sends back $h_1, \dots, h_m$ to the client, who stores the preprocessing $\text{cor} = (S, \Delta_1, \dots, \Delta_m, h_1, \dots, h_m)$. Note that computational overhead, the communication size as well as the size of the preprocessing are all $\text{poly}(\lambda) \cdot \sqrt{N}$. In the online phase, the client gets as input an index $x \in [N]$ and proceeds as follows. First, it chooses a set S_i such that $x \in S_i$. Such a set S_i exists as the S_i covers all of $[N]$. Next, he punctures the set S_i at x , i.e., he computes $S' = S_i \setminus \{x\}$ and sends the *explicit representation* of S' to the online server. The online server computes the parity $h' = \sum_{j \in S'} \text{DB}[j]$ and sends h' back to the client. The client can now compute

$$h_i - h' = \sum_{j \in S_i} \text{DB}[j] - \sum_{j \in S_i \setminus \{x\}} \text{DB}[j] = \text{DB}[x]$$

and thus retrieve the desired database entry $\text{DB}[x]$. In terms of computation and communication complexity of the online phase, observe that both are $\text{poly}(\lambda) \cdot \sqrt{N}$.

In terms of security, we would like to argue that the set S' reveals no information about x to the online server. However, upon a moment's reflection it becomes evident that this is not the case. Specifically, in the above construction, the element x is never contained in the set S' , so the online server learns *with certainty* that $x \notin S'$. The CK20 Protocol overcomes this problematic dependency of S' on the index x by occasionally introducing errors. Specifically, with probability $\approx 1/\sqrt{N}$ the set S_i is punctured at random index $x' \neq x$, so that with probability $\approx 1/\sqrt{N}$ x appears in S' , as it would in a randomly chosen set of size $n - 1$. In fact, it is shown in [21] that in the view of the online server, the set S' is distributed uniformly random and, in particular, independent of the index x . In the main body, we will use an abstraction we call *covering set generators* for the main technique of [21] of sampling correlated sets $S_1, \dots, S_m$. For details, refer to Section 2.4.

CK20 In Reverse. As discussed above, we adapt the preprocessing model of [21] to the setting of laconic cryptography, that is, in the online phase, only a succinct message is sent from the client, who now acts as a sender to the server, who is now the receiver. In the end, only the server obtains an output.

Specifically, assume the client/sender gets inputs $x \in [N]$ and $v, w \in \{0, 1\}$ in the online phase, and we would like the online server/receiver to learn $y = v \cdot \text{DB}[x] + w$, but nothing else about x, v, w . We will henceforth refer to the client only as the sender and to the online server only as the receiver.

The natural starting point for this is to run the CK20 protocol *in reverse*. Assume for now that $v = 1$, i.e., the sender wants to let the receiver learn

$y = \text{DB}[x] + w$, but nothing else about x and w . The protocol idea to achieve this is pretty simple: Assume we have the same preprocessing as in the CK20 protocol; hence the sender chooses a set S_i that contains x , computes $S' = S_i \setminus \{x\}$ and uses the corresponding hint h_i as a mask for w , i.e., he computes $u = w + h_i$. He then sends S' and u to the receiver. The receiver now computes

$$\begin{aligned} y &= u - \sum_{j \in S'} \text{DB}[j] \\ &= w + \sum_{j \in S_i} \text{DB}[j] - \sum_{j \in S_i \setminus \{x\}} \text{DB}[j] \\ &= w + \text{DB}[x], \end{aligned}$$

and obtains the desired output. Note that if we introduce errors with a small probability, as in the CK20 protocol, then the set S' is independent of x in the view of the receiver. To see that nothing beyond $y = w + \text{DB}[x]$ is revealed about w and x to the receiver, note that we can perfectly simulate the term u given just y , DB and S' by setting $u = y + \sum_{j \in S'} \text{DB}[j]$. Finally, also note that this protocol fulfills the laconicity requirement: In the online phase, only a single message of size $\text{poly}(\lambda) \cdot \sqrt{N}$ is sent, furthermore, both sender and receiver run in time $\text{poly}(\lambda) \cdot \sqrt{N}$. Moreover, as we use the same offline phase as the CK20 protocol, both computation and communication in the offline phase are of size $\text{poly}(\lambda) \sqrt{N}$.

To also allow for $v = 0$, one may consider the following naive approach: If $v = 1$, proceed as above, but if $v = 0$ *don't* puncture S_i at x , i.e., just send $S' = S_i$ and $u = w + h_i$ to the receiver. In terms of functionality, it routinely follows that the receiver obtains the correct output $y = w + v \cdot \text{DB}[x]$. However, in the case of $v = 0$, the sender now sends a set S' of cardinality n instead of $n - 1$ to the receiver, and hence, just by observing the cardinality of S' , the receiver can determine whether $v = 0$ or $v = 1$!. To address this issue, we can include a hint h^* for a uniformly random set S^* of size $n - 1$ into the preprocessing. If $v = 0$, the sender sets $S' = S^*$ and $u = w + h^*$. This now allows us to establish security, and in case S' does not contain an error, the receiver indeed correctly recovers $y = w + v \cdot \text{DB}[x]$. However, if the set S' does contain an error, the receiver gets a wrong output $y' = w + v \cdot \text{DB}[x']$ for some $x' \neq x$.

Note that errors are a much bigger issue in this protocol than in the CK20 protocol, as the sender cannot just tell the receiver that an error happened and the output is to be discarded as this would compromise index privacy, i.e., the effect of introducing errors would be gone.

Private Laconic Vector OLE. Instead of fixing the above protocol via an ad-hoc solution, we consider a generalization of the laconic OT functionality, which, at the same time, enables us to deal with errors in a conceptually clean and very efficient manner. The basic idea is simple: Instead of computing hints as *parities*, we compute them as *sparse linear functions*. Assume in the following that we are working over a finite field $\mathbb{F}$, specifically the database DB is in $\mathbb{F}^N$, and $\mathbf{v}, \mathbf{w} \in \mathbb{F}^k$

are $\mathbb{F}$ -vectors. In the following, we will use the following shorthand notation. For a set $S = \{s_1, \dots, s_n\}$ and a database $\text{DB} \in \mathbb{F}^N$, denote by $\text{DB}[S]$ the vector

$$\text{DB}[S] = \begin{pmatrix} \text{DB}[s_1] \\ \vdots \\ \text{DB}[s_n] \end{pmatrix} \in \mathbb{F}^n,$$

where we assume without loss of generality that the $s_1, \dots, s_n$ are sorted increasingly.

We want to securely compute the function $\mathbf{y} = \mathbf{w} + \mathbf{v} \cdot \text{DB}[x]$, i.e. the receiver obtains the output $\mathbf{y}$ but learns nothing else about $\mathbf{v}, \mathbf{w}$ and $x \in [N]$. To facilitate this, we modify the offline phase as follows. The receiver generates the sets $S_1, \dots, S_m$ as before, but additionally chooses a uniformly random matrix $\mathbf{R} \in \mathbb{F}^{(k+\lambda) \times n}$. The receiver sends the sets $S_1, \dots, S_m$ (in their compressed form) as well as the matrix $\mathbf{R}$ to the offline server. For $i \in [m]$ the offline server computes the hints

$$\mathbf{h}_i = \mathbf{R} \cdot \text{DB}[S_i]$$

The offline server then sends back the hints $\mathbf{h}_1, \dots, \mathbf{h}_m$ to the sender, who stores them, together with the matrix $\mathbf{R}$ and the sets $S_1, \dots, S_m$ (in their compressed form) for future use in the online phase.

In the online phase, the sender gets inputs $x \in [N]$ and $\mathbf{v}, \mathbf{w} \in \mathbb{F}^k$. As a warm-up, we will first consider the error-free case. As in the previous protocol, the sender chooses a set S_i which contains x and sets $S' = S_i \setminus \{x\}$. Now assume that x appears at position j^* in $S_i = \{s_{i,1}, \dots, s_{i,n}\}$, i.e. it holds that $s_{i,j^*} = x$. Let $\mathbf{R}_{-j^*} \in \mathbb{F}^{(k+\lambda) \times (n-1)}$ the matrix $\mathbf{R}$ with the j^* -th column $\mathbf{r}_{j^*}$ removed.

Finally, the sender chooses a uniformly random full-rank matrix $\mathbf{M} \in \mathbb{F}^{k \times (k+\lambda)}$ under the constraint that $\mathbf{M} \cdot \mathbf{r}_{j^*} = \mathbf{v}$, where $\mathbf{r}_{j^*}$ is the j^* -th column we've removed from $\mathbf{R}$. Note that the matrix $\mathbf{M}$ is *slightly shrinking* from $k + \lambda$ to k dimensions to provide a few additional degrees of freedom.

Finally, the sender sets $\mathbf{Z} = \mathbf{M} \cdot \mathbf{R}_{-j^*}$, and sends S' , $\mathbf{Z}$ and $\mathbf{u} = \mathbf{w} + \mathbf{M} \cdot \mathbf{h}_i$ (where $\mathbf{h}_i$ is the hint corresponding to S_i) to the receiver. The receiver now computes

$$\mathbf{y} = \mathbf{u} - \mathbf{Z} \cdot \text{DB}[S'] = \mathbf{w} + \mathbf{M} \cdot \mathbf{R} \cdot \text{DB}[S_i] - \mathbf{M} \cdot \mathbf{R}_{-j^*} \text{DB}[S'].$$

Now note that since $S' = S_i \setminus \{x\}$ it holds that

$$\mathbf{R} \cdot \text{DB}[S_i] = \mathbf{R}_{-j^*} \text{DB}[S'] + \mathbf{r}_{j^*} \cdot \text{DB}[x].$$

Consequently, it holds that

$$\mathbf{y} = \mathbf{w} + \mathbf{M} \cdot \mathbf{r}_{j^*} \cdot \text{DB}[x] = \mathbf{w} + \mathbf{v} \cdot \text{DB}[x]$$

as $\mathbf{M}$ was chosen such that $\mathbf{M} \cdot \mathbf{r}_{j^*} = \mathbf{v}$. Hence, the receiver obtains the desired output.

The key property we exploited in this construction is that we can *program* a matrix $\mathbf{M}$ which maps the missing vector $\mathbf{r}_{j^*}$ to $\mathbf{v}$. To argue security, we will

need to take errors into account. So far, we have just used *one degree of freedom* of programming the matrix $\mathbf{M}$, but there are still $\lambda - 1$ degrees of freedom left. In the next paragraph, we will use the remaining degrees of freedom to *erase* erroneous positions by choosing $\mathbf{M}$ in such a way that it maps the corresponding $\mathbf{r}_{j^*}$ to 0.

The Full Protocol: Erasing Errors. In the full protocol, we will repeat the core component of the protocol sketched in the last paragraph λ times in parallel but also introduce errors into the sets S' .

Specifically, assume we have λ preprocessings, that is the receiver generates λ sets $S'_1, \dots, S'_\lambda$, λ matrices $\mathbf{R}_1, \dots, \mathbf{R}_\lambda$ as well as λ indices $j_1^*, \dots, j_\lambda^*$ and retrieves the corresponding hints $\mathbf{h}_1, \dots, \mathbf{h}_\lambda$. Each of the S'_ℓ (for $\ell \in [\lambda]$) is *faulty* with probability $1/\sqrt{N}$. That is, the faulty S'_ℓ were punctured at points $x'_\ell \neq x$.

We will now harness the fact that the sender knows which indices ℓ are faulty. Specifically, it will suffice to have a single index ℓ for which S'_ℓ is not faulty, i.e., S'_ℓ was punctured at the correct point x . Such an index ℓ_0 must exist, except with probability $(1/\sqrt{N})^\lambda$, which is negligible. Hence, fix such an ℓ_0 . Now let $\mathbf{r}_{\ell, j_\ell^*}$ be the j_ℓ^* -th column of $\mathbf{R}_\ell$. The receiver chooses a uniformly random full-rank matrix $\mathbf{M}$ under the constraint that $\mathbf{M}\mathbf{r}_{\ell_0, j_{\ell_0}^*} = \mathbf{v}$ and $\mathbf{M}\mathbf{r}_{\ell, j_\ell^*} = 0$ for all $\ell \in [\lambda] \setminus \{\ell_0\}$. The sender now set $\mathbf{Z}_\ell = \mathbf{M} \cdot \mathbf{R}_{\ell, -j_\ell^*}$ for $\ell \in [\lambda]$ and further set $\mathbf{u} = \mathbf{w} + \mathbf{M} \cdot \left(\sum_{\ell \in [\lambda]} \mathbf{h}_\ell\right)$. He sends the S'_ℓ , the $\mathbf{Z}_\ell$ and $\mathbf{u}$ to the receiver. The receiver computes and outputs

$$\mathbf{y} = \mathbf{u} - \sum_{\ell \in [\lambda]} \mathbf{Z}_\ell \text{DB}[S'_\ell]$$

Note the analysis above it holds for all ℓ that

$$\mathbf{h}_\ell = \mathbf{R}_{\ell, -j_\ell^*} \cdot \text{DB}[S'_\ell] + \mathbf{r}_{\ell, j_\ell^*} \cdot \text{DB}[x'_\ell].$$

Hence, by a similar analysis as in the last paragraph, we can write $\mathbf{y}$ as

$$\mathbf{y} = \mathbf{w} + \sum_{\ell \in [\lambda]} \mathbf{M} \cdot \mathbf{r}_{\ell, j_\ell^*} \cdot \text{DB}[x'_\ell].$$

Now recall for $\ell_0 \in [\lambda]$ it holds that $x'_{\ell_0} = x$ and $\mathbf{M}\mathbf{r}_{\ell_0, j_{\ell_0}^*} = \mathbf{v}$, and for all other ℓ that $\mathbf{M}\mathbf{r}_{\ell, j_\ell^*} = 0$. Hence, we obtain $\mathbf{y} = \mathbf{w} + \mathbf{v} \cdot \text{DB}[x]$, and conclude that the protocol is correct.

To argue security, recall that by the analysis of [21], the sets S'_ℓ are independent of x . We can further argue that the matrices $\mathbf{Z}_\ell$ are uniform in the view of the receiver. Hence we can simulate $\mathbf{u}$ via

$$\mathbf{u} = \mathbf{y} + \sum_{\ell \in [\lambda]} \mathbf{Z}_\ell \text{DB}[S'_\ell],$$

and conclude that this protocol satisfies sender privacy, i.e., the sender's message reveals nothing more than $\mathbf{y}$.

Receiver Privacy The protocol, as sketched above, does not provide receiver privacy, as the hints $\mathbf{h}_i$ leak non-trivial information about the database DB to the sender. To make this protocol receiver-private, we will mask each hint with a pseudorandom value and provide the corresponding PRF key to the receiver to give him the ability to unmask the value $\mathbf{u}$. This will further make it necessary to also send the matrix $\mathbf{M}$ to the receiver, which in particular means that we have to ensure that $\mathbf{M}$ does not leak secret information to the receiver. However, the way we sampled $\mathbf{M}$ ensures that its marginal distribution is uniform, i.e., it does not leak information about x , $\mathbf{v}$ and $\mathbf{w}$. For detail see Lemma 3.

Laconic Private Set Intersection with Preprocessing. We will briefly discuss our main application of our private laconic VOLE protocol, a laconic PSI protocol in the preprocessing model.

$\mathbb{F} = \mathbb{F}_{2^{2\lambda}}$ be the finite field of size $2^{2\lambda}$. Without loss of generality, assume we are working over the universe $\mathcal{U} = \{0, 1\}^\lambda$. Any larger universe can be mapped to $\mathcal{U}$ via a collision-resistant hash function. Furthermore, there is a bijective function $\varphi : \{0, 1\}^{2\lambda} \rightarrow \mathbb{F}$ which allows us to embed elements of $\mathcal{U}$ into $\mathbb{F}$ by e.g. sending a string $x \in \mathcal{U}$ to the field element $\varphi(x, 0^\lambda)$. We consider the setting of private set membership (PSM), where the sender just holds a singleton set $Y = \{y\}$. Any PSM protocol immediately implies a PSI protocol by running the PSM protocol for every element in the sender's set.

Now let $X \subseteq \mathcal{U}$ be the receiver's set. The offline server first constructs a λ databases $\text{DB}_1, \dots, \text{DB}_\lambda \in \mathbb{F}^N$, where $N = \lambda \cdot |X|$. He chooses λ pairwise independent hash functions $f_1, \dots, f_\lambda$, where $f_\ell : \mathcal{U} \rightarrow [N]$. Next, he constructs DB_ℓ in such a way that for each index $j \in [N]$ it holds that either $\text{DB}_\ell[j] = 0$, or $\text{DB}_\ell[j] = \varphi(x, 0^\lambda)$, where $x \in X$ is such that $f_\ell(x) = j$. We can construct such a database by just setting $\text{DB}_\ell[f_\ell(x)] = \varphi(x, 0^\lambda)$, and overwrite if a collision occurs. A simple analysis shows that if we construct the DB_ℓ in this way, then for every $x \in X$ there exists an $\ell \in [\lambda]$ such that $\text{DB}_\ell[f_\ell(x)] = \varphi(x, 0^\lambda)$, except with negligible choice over the pairwise independent hash functions $f_1, \dots, f_\lambda$. We then compute preprocessing for all databases $\text{DB}_1, \dots, \text{DB}_\lambda$, and include the $f_1, \dots, f_\lambda$ into the preprocessing of both sender and receiver. Observe that the f_ℓ are chosen uniformly; hence, they do not leak any information about X .

In the online phase, let $y \in \mathcal{U}$ be the input of the sender. The sender now chooses λ random field elements $\mathbf{r}_1, \dots, \mathbf{r}_\lambda$, and prepares plvOLE instances, where the ℓ -th instance is with respect to database DB_ℓ gets as sender input an index $i_\ell = f_\ell(y)$, and vectors $\mathbf{v}_\ell = -\mathbf{r}_\ell$ and $\mathbf{w}_\ell = (\mathbf{r}_\ell + 1) \cdot \varphi(y, 0^\lambda)$. Consequently, the receiver gets outputs plvOLE outputs $\mathbf{y}_1, \dots, \mathbf{y}_\ell$ with

$$\mathbf{y}_\ell = \mathbf{w}_\ell + \mathbf{v}_\ell \text{DB}_\ell[f_\ell(y)] = \varphi(y, 0^\lambda) + \mathbf{r}_\ell \cdot (\varphi(y, 0^\lambda) - \text{DB}_\ell[f_\ell(y)]).$$

First, consider the case that $y \in X$. In this case, by the above discussion, there exists an index $\ell \in [\lambda]$ for which $\text{DB}_\ell[f_\ell(y)] = \varphi(y, 0^\lambda)$. Hence, for this index ℓ , the receiver will get an output $\mathbf{y}_\ell = \varphi(y, 0^\lambda)$, detect that this value is correctly padded with 0^λ and output y .

One the other hand, if $y \notin X$, then for all ℓ it holds that $\text{DB}_\ell[f_\ell(y)]$ is distinct from $\varphi(y, 0^\lambda)$. Consequently all $\varphi(y, 0^\lambda) - \text{DB}_\ell[f_\ell(y)]$ are non-zero. But

this implies that all $\mathbf{y}_\ell$ are distributed uniformly and independently of y , as the $\mathbf{r}_\ell$ are distributed uniformly. We conclude that in this case, the receiver gets no information about y .

This concludes this outline.

2 Preliminaries

2.1 Notation

We use λ to denote the security parameter. We use bold lower-case letters such as $\mathbf{v}$ to represent row vectors, v_i to denote the i -th coordinate of $\mathbf{v}$, and the punctured vector $\mathbf{v}_{-i}$ to represent the vector identical to $\mathbf{v}$ with the exception that $v_i = 0$.

We use $\mathbb{N}$ to denote the set of positive integers. We denote by $[n]$ the set $\{1, \dots, n\}$ for an integer $n \in \mathbb{N}$, and by $[a, b]$ the set $\{a, \dots, b\}$ for $a, b \in \mathbb{N}$ and $a < b$. We use $\parallel$ to denote a concatenation of two binary strings.

We denote by $\text{negl}(\cdot)$ a negligible function in its parameter, and $\text{poly}(\cdot)$ a fixed polynomial in its parameter. In this work, logarithms are taken to the base 2, and we treat expressions, e.g., $\sqrt{n}$, $\log n$, and m/n as integers, ignoring integrality.

2.2 Probability

Lemma 1. *Let $X_1, \dots, X_n$ be i.i.d. Poisson trials, i.e. it holds that X_i is 1 with probability p and otherwise 0. Then it holds for every $\delta > 0$ that*

$$\Pr\left[\sum_{i=1}^n X_i \geq (1 + \delta)pn\right] < e^{-\delta^2 \cdot p \cdot n/3}.$$

2.3 Linear Algebra

The following standard Lemma establishes that t randomly chosen vectors in a vector-space of dimension slightly larger than t are linearly independent, except with negligible probability.

Lemma 2. *Let $\mathbb{F}$ the finite field of size q . Let $\mathbf{a}_1, \dots, \mathbf{a}_t \in \mathbb{F}^n$ be chosen uniformly at random, where $t < k + \lambda$. Then the $\mathbf{a}_1, \dots, \mathbf{a}_t$ are linearly independent, except with probability*

$$1 - \prod_{i=1}^t (1 - q^{i-n-1}) \leq \sum_{i=1}^t q^{i-n-1} \leq t \cdot q^{t-n-1}$$

We will also need the following simple linear algebra fact.

Lemma 3. Assume that $n \geq k + t$. Let $\mathbf{a}_1, \dots, \mathbf{a}_t \in \mathbb{F}^n$ be distributed uniformly random under the constraint that they are linearly independent, and let $\mathbf{b}_1, \dots, \mathbf{b}_t \in \mathbb{F}^k$ be arbitrary. Then we can efficiently sample a uniformly random full-rank matrix $\mathbf{M} \in \mathbb{F}^{k \times n}$ such that it holds that $\mathbf{M}\mathbf{a}_i = \mathbf{b}_i$ for all $i \in [t]$. Furthermore, the marginal distribution of $\mathbf{M}$ is uniform over all full-rank matrices in $\mathbb{F}^{k \times n}$.

Proof. Fix any vectors $\mathbf{b}_{t+1}, \dots, \mathbf{b}_{t+k}$ that form a basis of $\mathbb{F}^k$, and set $\mathbf{b}_{t+k+j}$ for $j \in [n - t - k]$ to 0. This is always possible as $n \geq k + t$. Set $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n) \in \mathbb{F}^{k \times n}$. Since the $\mathbf{a}_1, \dots, \mathbf{a}_t$ uniform under the constraint of being linearly independent, we can extend them to a uniformly random basis $\mathbf{a}_1, \dots, \mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_n$ of $\mathbb{F}^n$ by choosing $\mathbf{a}_{t+1}, \dots, \mathbf{a}_n \in \mathbb{F}^n$ uniformly random under this constraint. Hence the matrix $\mathbf{A} = (\mathbf{a}_1, \dots, \mathbf{a}_n) \in \mathbb{F}^{n \times n}$ is a uniformly random invertible matrix, i.e. it is uniformly random in the group $\text{GL}_n(\mathbb{F})$. We can thus set $\mathbf{M} = \mathbf{B} \cdot \mathbf{A}^{-1}$. Observe that $\mathbf{M} \cdot \mathbf{A} = \mathbf{B}$, i.e. $\mathbf{M}$ fulfills the required constraints. Finally, since $\mathbf{B}$ is full rank and $\mathbf{A}^{-1}$ is a uniformly random invertible matrix, $\mathbf{M}$ is a uniformly random full-rank matrix in $\mathbb{F}^{k \times n}$.

2.4 Covering Set Generators

We will use an abstraction we call *covering set generators*, which abstracts-out the main techniques used by Corrigan-Gibbs and Kogan [21] to construct doubly efficient private information retrieval in the preprocessing model.

Definition 4. Let $N = N(\lambda), m = m(\lambda), n = n(\lambda)$ be positive integers with $N > n$, and let $\delta = \delta(\lambda) \in [0, 1]$. A (N, m, n, δ) -covering set generator $\text{SetGen} = (\text{Gen}, \text{SampleSet})$ consists of two PPT algorithms with the following syntax.

- $\text{Gen}(1^\lambda)$ takes as input a security parameter and outputs a seed $\text{seed} \in \{0, 1\}^\lambda$ and a collection of m sets $(S_1, \dots, S_m)$, where each $S_i \subseteq [N]$ is of size n
- $\text{SampleSet}(\text{seed}, x)$ takes a seed seed and an element $x \in [N]$ and outputs a set $S' \subseteq [N]$ of size $n - 1$, an element $x' \in [N]$ and an index $i \in [m]$.

We require that the following properties hold.

Correctness : For all λ and for every index $x \in [N]$, given that $(\text{seed}, (S_i)_{i \in [m]}) \leftarrow \text{Gen}(1^\lambda)$ and $(S', x', i) \leftarrow \text{SampleSet}(\text{seed}, x)$ we require the following:

1. It holds that $S' \cup \{x'\} = S_i$, except with negligible probability over all random choices.
2. It holds that $x' = x$, except with probability δ over all random choices.

We refer to δ as the error probability of the covering set generator.

Hiding : There exists a PPT simulator $\text{Sim}_{\text{SetGen}}$ such that for all λ and all $x \in [N]$ it holds that

$$(S', i) \approx_{s/c} \text{Sim}_{\text{SetGen}}(1^\lambda),$$

where $(\text{seed}, \cdot) \leftarrow \text{Gen}(1^\lambda)$ and $(S', x', i) \leftarrow \text{SampleSet}(\text{seed}, x)$. Here indistinguishability can be statistical or computational. We have used the $\cdot$ notation to denote that an output is omitted.

We will briefly sketch the construction of covering set generators implicit in [21]. Their construction can be outlined as follows. Let λ be a statistical security parameter. The database size $N = N(\lambda)$ is a free parameter which we think of as a large polynomial in λ . The set size n is typically chosen as $\sqrt{N}$, though different choices with different tradeoffs are possible. The number of sets m is chosen as $\lambda\sqrt{N}$.

- The generator function chooses a uniformly random set $S \subseteq [N]$ of size n and m random shifts $\Delta_i \in [N]$ for $i = 1, \dots, m$. It sets $\text{seed} = (S, (\Delta_i)_{i \in [m]})$ and $S_i = \{s + \Delta_i \bmod N \mid s \in S\} \subseteq [N]$ for $i = 1, \dots, m$.
- Given an element $x \in [N]$, the sampler **SampleSet** finds an index i such that Δ_i such that $x - \Delta_i \bmod N \in S$ (and hence $x \in S_i$). It flips biased coin which comes out 0 with probability $1 - \delta$. If the coin comes out 0 it sets $S' = S_i \setminus \{x\}$ and $x' = x$. If the coin comes out 1 it chooses a uniformly random $x' \in S_i \setminus \{x\}$ and sets $S' = S_i \setminus \{x'\}$. It then outputs S' , x' and i .

The correctness property can be established routinely by relying on the uniformity of the Δ_i : Fix an $x \in [N]$. Then it holds for every $i \in [m]$ that $\Pr[x - \Delta_i \bmod N \notin S] = 1 - 1/n$ by the uniformity of Δ_i . As the Δ_i are independent, it holds that $\Pr[\forall i \in [m] : x - \Delta_i \bmod N \notin S] = (1 - 1/n)^m \leq e^{-m/n} \leq e^{-\lambda}$ as $m = \lambda \cdot n$. Consequently, a union bound over all $x \in [N]$ yields that the probability that there exists an $x \in [N]$ for which x is in none of the sets S_i is at most $N \cdot e^{-\lambda} = \text{negl}(\lambda)$. Hence the first item of the correctness property follows. The second item of the correctness property now follows immediately from the definition of the **SampleSet** algorithm, which introduces an error with probability δ .

Establishing the hiding property is a bit more involved, and this is where the choice of the error-probability δ is crucial. Corrigan-Gibbs and Kogan [21] show that for $\delta = (n - 1)/N \approx 1/\sqrt{N}$ the marginal distribution of the punctured set S' generated by **SampleSet** is statistically close to uniform, and thus in particular independent of the set element x . Note that as the shifts Δ_i are chosen uniformly random, the index i is also just uniformly random in $[m]$ given S' . Hence, the statistical simulator $\text{Sim}_{\text{SetGen}}$ just outputs a uniformly random set $S' \subset [N]$ of size $n - 1$ and a uniformly random index $i \leftarrow [m]$.

We remark that the seed can be compressed to λ bits, just define seed' to be the seed of a PRF which provides the random coins for the **Gen** algorithm.

2.5 Offline/Online Two-server Models

We model our laconic functionalities with preprocessing in the offline/online two-server setting. In this setting, there are three parties: a client, an offline server, and an online server. We refer to the online server that receives the function output as the receiver and the client as the sender. Both the offline and the online server hold a replica of a database DB , but they are not allowed to interact or collude, as established in the model in [6, 21].

During the offline phase, the offline server locally generates correlations or preprocessed data based on the database and sends them to the sender. In the

online phase, the sender uses these preprocessed correlations to interact with the receiver. The security of the model crucially relies on the offline server and the online server (the receiver) not communicating or colluding with each other.

2.6 Pseudorandom Functions

Definition 5 ([36]). A pseudorandom function $\text{PRF}_K : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is an efficiently computable keyed-function such that it holds for every PPT-oracle distinguisher $\mathcal{D}$ and all $\lambda \in \mathbb{N}$ that

$$|\Pr[\mathcal{D}^{\text{PRF}_K(\cdot)}(1^\lambda) = 1] - \Pr[\mathcal{D}^{F(\cdot)}(1^\lambda) = 1]| \leq \text{negl}(\lambda),$$

where $K \leftarrow \{0, 1\}^\lambda$ is a uniformly chosen key and $F : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is a uniformly random function.

2.7 Universal Hash Functions

Definition 6 (see e.g. [42]). Let $\mathcal{H}$ be a family of hash functions with domain $\mathcal{D}$ and range $\mathcal{R}$. Then $\mathcal{H}$ is a family of universal hash functions if for every distinct $x_1, x_2 \in \mathcal{D}$, $\mathcal{H}$ satisfies:

$$\Pr[h(x_1) = h(x_2) : h \leftarrow \mathcal{H}] \leq 1/|\mathcal{R}|$$

2.8 Garbled Circuits

Garbled circuits are an essential cryptographic tool in the study of secure two-party computation.

Definition 7 (Garbled Circuits). A garbling scheme consists of two algorithms (Garble, Eval) with the following syntax, correctness and security properties.

- $(C, \{\text{lb}_{i,b}\}) \leftarrow \text{Garble}(1^\lambda, C)$: Given the security parameter λ and circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}^n$, it outputs the garbled circuit C along with all the input labels of the garbled circuit $\{\text{lb}_{i,b}\}$.
- $y \leftarrow \text{Eval}(C, \{\text{lb}_i\})$: Given the garbled circuit C and a label for each input wire denoted by $\{\text{lb}_i\}$, it outputs a value y .

We also require the following properties to be satisfied:

Correctness: For all circuits C and input $x \in \{0, 1\}^\ell$:

$$\Pr[\text{Eval}(C, \{\text{lb}_{i,x[i]}\}) = C(x) \mid (C, \{\text{lb}_{i,b}\}) \leftarrow \text{Garble}(1^\lambda, C)] = 1$$

Security: For all circuits C and input $x \in \{0, 1\}^\ell$, there exists a simulator such that on input the security parameter λ and output value y and outputs a simulated garbled circuit $\hat{C}$ and corresponding simulated label values $\{\hat{\text{lb}}_{i,b}\}$, and it holds that:

$$\{(C, \{\text{lb}_{i,x[i]}\}) \mid (C, \{\text{lb}_{i,b}\}) \leftarrow \text{Garble}(1^\lambda, C)\} \stackrel{c}{\approx} \{(\hat{C}, \{\hat{\text{lb}}_{i,x[i]}\}) \mid (\hat{C}, \{\hat{\text{lb}}_{i,b}\}) \leftarrow \text{Sim}(1^\lambda, y)\}$$

3 Database-Dependent Preprocessing

In this part, we will argue that laconic OT with database-independent preprocessing, where the online phase has at most two rounds, implies a standard OT protocol. Hence, it is unlikely that there exists a laconic OT with database-independent preprocessing from symmetric assumptions alone. In particular, such a protocol would need to overcome classical and newer black-box impossibility results [30, 43].

We observe that we can strengthen this result by showing that it still holds even if the preprocessing step is allowed to use arbitrarily strong public key primitives. The reason is simple: In our transformed protocol, the preprocessing is entirely performed by the sender, i.e., the receiver runs none of the preprocessing algorithms. In other words, in the transformed protocol, the receiver still only uses symmetric key algorithms, which effectively means it does not matter whether the public key primitives used by the sender were secure or insecure, i.e., the transformed protocol would still have to be secure even if we used an insecure instantiation of these public key primitives.

We will establish these results in the following steps.

1. If the protocol we start with has receiver privacy, we will ignore this feature, i.e., consider it a protocol without receiver privacy.
2. Since the preprocessing phase is independent of the database and as there is no receiver privacy, we can collapse the preprocessing into a single message from the sender to the receiver.
3. We will argue that such a protocol, i.e., laconic OT with a one-message database-independent preprocessing, implies a standard OT protocol with *weak receiver security*.
4. Such a protocol can be amplified into an OT protocol with standard security against semi-honest senders.

Steps 1 and 2 are trivial. We will hence focus on Step 3 and Step 4.

In the following, since this is only a complementary result, we will assume familiarity with standard notions concerning oblivious transfer and forgo providing the most formal definitions.

As mentioned above, we will consider a weak notion of receiver privacy. Specifically, we will consider a setting where the receiver's choice bit is chosen at random, and security only guarantees that the receiver's choice bit is not fully determined by the view of the sender, i.e., the choice bit has some residual Shannon entropy given the view of the sender. A body of works considers the problem of constructing OT protocols with standard security from weakly secure protocols, e.g., [17]. In a nutshell, one can amplify residual Shannon entropy to min-entropy by parallel repetition and then extract a uniformly random choice bit via standard privacy amplification techniques. We will use the following notion of receiver privacy.

Definition 8 (Weak Receiver Privacy). *Let OT be an OT protocol with a sender S and a receiver R , and let $\phi > 0$. We say an OT protocol satisfies ϕ -weak*

receiver privacy, if $H(b|\text{view}(S)) \geq \phi$. Here

$$H(X|Y) = - \sum_{x,y} \Pr[X = x, Y = y] \log \left(\frac{\Pr[X = x, Y = y]}{\Pr[X = x]} \right)$$

is the conditional Shannon entropy.

We will finally turn to Step 3, constructing an OT protocol with weak receiver privacy. The idea of this transformation is simple: The receiver chooses a uniformly random database DB, and the sender chooses a random index i . As the first round laconic OT message IOT_1 is short, the entropy of DB cannot be reduced by a lot given IOT_1 , i.e., it cannot encode too much information about DB. Even though the first laconic OT message IOT_1 may somehow reveal the bit DB_i for a wrong choice of i , we can show that since i is chosen at random, it holds that the bit DB_i has high (Shannon) entropy *on average*.

We will thus focus on Step 3. Let $\text{IOT} = (\text{Offline}, \text{Rec}_1, \text{Send}, \text{Rec}_2)$ be a 2-message laconic OT protocol with *database-independent* preprocessing Offline. Consider the following OT protocol $\text{OT} = (\text{Send}_1, \text{Rec}_1, \text{Send}_2, \text{Rec}_2)$.

- $\text{Send}_1(1^\lambda, n)$: Compute $(\text{st}_{\text{Send}}, \text{st}_{\text{Rec},1}) = \text{Offline}(1^\lambda, n)$. Output st_{Send} and $\text{st}_{\text{Rec},1}$.
- $\text{Rec}_1(\text{st}_{\text{Rec},1})$: Choose $\text{DB} \leftarrow \{0,1\}^n$ uniformly at random. Compute $(\text{IOT}_1, \text{st}_{\text{Rec},2}) = \text{IOT}.\text{Rec}_1(\text{st}_{\text{Rec},1}, \text{DB})$ and output IOT_1 and $\text{st}_{\text{Rec},2}$.
- $\text{Send}_1(\text{st}_{\text{Send}}, m_0, m_1)$: Choose $i \leftarrow [n]$ uniformly at random. Compute and output $\text{IOT}_2 = \text{IOT}.\text{Send}(\text{st}_{\text{Send}}, m_0, m_1)$.
- $\text{Rec}_2(\text{st}_{\text{Rec},2}, \text{IOT}_2)$: Compute $b = \text{DB}_i$ and $m^* = \text{IOT}.\text{Rec}_2(\text{st}_{\text{Rec},2}, \text{IOT}_2)$. Output b, m^* .

We will briefly and informally argue the correctness and sender-privacy of this protocol. Clearly, by the correctness of IOT it holds that $m^* = m_{\text{DB}_i} = m_b$. Moreover, by the sender-privacy of IOT it holds that IOT_2 can be simulated given only $m^* = m_{\text{DB}_i} = m_b$.

We will now establish weak receiver security of our protocol above.

Lemma 1 *The protocol OT above satisfies $1 - \frac{k}{n}$ weak receiver security.*

Proof. To establish weak receiver privacy, we must show that the receiver's choice bit $b = \text{DB}_i$ has non-trivial Shannon entropy given the sender's view. The view of the sender consists of the database-independent preprocessing st_{Send} , as well as the first laconic OT message IOT_1 . By the Shannon-entropy chain rule, it holds that

$$\begin{aligned} H(\text{DB}|\text{IOT}_1, \text{st}_{\text{Send}}) &= H(\text{DB}, \text{IOT}_1|\text{st}_{\text{Send}}) - H(\text{IOT}_1|\text{st}_{\text{Send}}) \\ &\geq H(\text{DB}) - k \\ &= n - k \end{aligned}$$

as st_{Send} is independent of DB and $|\text{IOT}_1| \leq k$.

By the Shannon-entropy chain rule, it holds that

$$\begin{aligned}
 H(\text{DB}|\text{IOT}_1, \text{st}_{\text{Send}}) &= H(\text{DB}_1, \dots, \text{DB}_n|\text{IOT}_1, \text{st}_{\text{Send}}) \\
 &= \sum_{j=1}^n H(\text{DB}_j|\text{DB}_1, \dots, \text{DB}_{j-1}, \text{IOT}_1, \text{st}_{\text{Send}}) \\
 &\leq \sum_{j=1}^n H(\text{DB}_j|\text{IOT}_1, \text{st}_{\text{Send}}) \\
 &= n \cdot \sum_{j=1}^n \frac{1}{n} H(\text{DB}_j|\text{IOT}_1, \text{st}_{\text{Send}}) \\
 &= n \cdot \sum_{j=1}^n \Pr[i = j] H(\text{DB}_i|\text{IOT}_1, \text{st}_{\text{Send}}, i = j) \\
 &= n \cdot H(\text{DB}_i|\text{IOT}_1, \text{st}_{\text{Send}}, i).
 \end{aligned}$$

Hence, we obtain that

$$\begin{aligned}
 H(\text{DB}_i|\text{IOT}_1, \text{st}_{\text{Send}}, i) &\geq \frac{1}{n} \cdot H(\text{DB}|\text{IOT}_1, \text{st}_{\text{Send}}) \\
 &\geq \frac{n-k}{n} = 1 - \frac{k}{n}.
 \end{aligned}$$

□

4 Private Laconic Vector-OLE with Preprocessing

In this Section, we will provide our main construction, a private laconic vector-OLE with preprocessing protocol. We will start by providing a definition of this primitive.

Definition 9 (Private Laconic Vector OLE with Preprocessing). *Syntactically, a private laconic vector-OLE scheme plvOLE consists of three algorithms (Offline, Send, Receive) with the following syntax. In the following, let $\mathbb{F}$ be a finite field (potentially depending on λ).*

- **Offline**($1^\lambda, k, \text{DB}$): Given a database $\text{DB} \in \mathbb{F}^N$, it outputs a sender preprocessing cor_S and a receiver preprocessing cor_R .
- **Send**($\text{cor}_S, x, \mathbf{v}, \mathbf{w}$): Given a sender preprocessing cor_S , a database location $x \in [N]$ and two vectors $\mathbf{v}, \mathbf{w} \in \mathbb{F}^k$ of length k , it outputs a sender-message $\mathbf{c}$.
- **Receive**($\text{cor}_R, \text{DB}, \mathbf{c}$): Given a receiver preprocessing cor_R , the database DB and a sender-message $\mathbf{c}$, it outputs a vector $\mathbf{y} \in \mathbb{F}^k$.

We require the following properties.

Correctness: For all $\lambda \in \mathbb{N}$, $N = \text{poly}(\lambda)$, any database $\text{DB} \in \mathbb{F}^N$, any $x \in [N]$ and all vectors $\mathbf{v}, \mathbf{w} \in \mathbb{F}^k$, it holds that

$$\text{Receive}(\text{cor}_R, \text{DB}, \text{Send}(\text{cor}_S, x, \mathbf{v}, \mathbf{w})) = \mathbf{v} \cdot \text{DB}[x] + \mathbf{w},$$

except with negligible probability over the random choices involved, where $(\text{cor}_S, \text{cor}_R) = \text{Offline}(1^\lambda, k, \text{DB})$.

Compactness and Efficiency: We require that:

- The bit-sizes of cor_S and cor_R are $o(N) \cdot \text{poly}(\lambda)$, i.e. they are sublinear in N
- The bit-size of c is $o(N) \cdot \text{poly}(\lambda)$, i.e. it is sublinear in N .
- Both Send and Receive run in time $o(N) \cdot \text{poly}(\lambda)$, i.e. sublinear in N .

Sender Privacy Against Semi-honest Receivers: There exists a PPT simulator Sim_R such that for all $\lambda \in \mathbb{N}$, $N = \text{poly}(\lambda)$, any database $\text{DB} \in \mathbb{F}^N$, any location $x \in [N]$ and any pair of vectors $\mathbf{v}, \mathbf{w} \in \mathbb{F}^k$ it holds that

$$(\text{cor}_R, \text{Send}(\text{cor}_S, x, \mathbf{v}, \mathbf{w})) \approx_{s/c} \text{Sim}_R(1^\lambda, N, \mathbf{v} \cdot \text{DB}[x] + \mathbf{w}),$$

where $(\text{cor}_S, \text{cor}_R) = \text{Offline}(1^\lambda, k, \text{DB})$. Here the indistinguishability can be statistical or computational.

Receiver Privacy against Semi-honest Senders: There exists a PPT simulator Sim_S such that for all $\lambda \in \mathbb{N}$, $N = \text{poly}(\lambda)$, and any database $\text{DB} \in \mathbb{F}^N$ it holds that

$$\text{cor}_S \approx_{s/c} \text{Sim}_S(1^\lambda, N)$$

where $(\text{cor}_S, \text{cor}_R) = \text{Offline}(1^\lambda, k, \text{DB})$. Here the indistinguishability can be statistical or computational.

4.1 Construction

We will now provide our construction of plvOLE . As a building block, we will use a *covering set generator*. The following construction already relies on a parallel repetition to handle the errors resulting from the covering set generator, and we use the variable $\ell \in [\lambda]$ to denote the index of the parallel repetition.

Construction 1 (Private Laconic Vector OLE with Preprocessing) Let $\mathbb{F}$ be any field and let $k = k(\lambda)$ be an integer, concretely k is the VOLE-dimension. Let $N = N(\lambda), m = m(\lambda), n = n(\lambda) \in \mathbb{N}$ be integers with $n < N$. Let $\text{PRF}_K : [\lambda] \times [m] \times [N] \rightarrow \mathbb{F}^{k+\lambda}$ and $\text{PRF}'_{K'} : [\lambda] \times [m] \rightarrow \mathbb{F}^{k+\lambda}$ be a pseudorandom function families, and let $\text{SetGen} = (\text{Gen}, \text{SampleSet})$ be a (N, m, n, δ) -covering set generator. The protocol $\text{plvOLE} = (\text{Offline}, \text{Send}, \text{Receive})$ is given as follows.

$\text{Offline}(1^\lambda, k, \text{DB}) :$

- For $\ell \in [\lambda]$, compute $(\text{seed}_\ell, (S_{\ell,i})_{i \in [m]}) \leftarrow \text{Gen}(1^\lambda)$
- Choose a PRF keys $K \leftarrow_{\$} \{0, 1\}^\lambda$ and $K' \leftarrow_{\$} \{0, 1\}^\lambda$
- For $\ell \in [\lambda]$, $i \in [m]$, compute

$$h_{\ell,i} \leftarrow \text{PRF}'_{K'}(\ell, i) + \sum_{j \in S_{\ell,i}} \text{PRF}_K(\ell, i, j) \cdot \text{DB}[j].$$

- Output $\text{cor}_S \leftarrow (K, (\text{seed}_\ell, (\mathbf{h}_{\ell,i})_{i \in [m]})_{\ell \in [\lambda]})$ and $\text{cor}_R = K'$
- Send($\text{cor}_S = (\text{seed}_\ell, K, (\mathbf{h}_{\ell,i})_{i \in [m]})_{\ell \in [\lambda]}, x, \mathbf{v}, \mathbf{w}$) :
- For $\ell \in [\lambda]$, compute $(S'_\ell, x'_\ell, i_\ell) \leftarrow \text{SampleSet}(\text{seed}_\ell, x)$.
 - Set $L = \{\ell \in [\lambda] \mid x'_\ell = x\}$ be the set of repetition-indices ℓ where no error occurred.
 - Choose a uniformly random full-rank matrix $\mathbf{M} \in \mathbb{F}^{k \times (k+\lambda)}$ under the constraint that

$$\mathbf{M} \cdot \left(\sum_{\ell \in L} \text{PRF}_K(\ell, i_\ell, x'_\ell) \right) = \mathbf{v}$$

and for all $\ell \in [\lambda] \setminus L$

$$\mathbf{M} \cdot \text{PRF}_K(\ell, i_\ell, x'_\ell) = \mathbf{0}.$$

If no such matrix $\mathbf{M}$ exists, output $\perp$.

- Compute $\mathbf{u} = \mathbf{w} + \mathbf{M} \cdot (\sum_{\ell \in [\lambda]} \mathbf{h}_{\ell,i_\ell})$.
 - For $\ell \in [\lambda]$, compute $Z_\ell = (\mathbf{M} \cdot \text{PRF}_K(\ell, i_\ell, j))_{j \in S'_\ell}$
 - Output $\mathbf{c} = ((S'_\ell, Z_\ell, i_\ell)_{\ell \in [\lambda]}, \mathbf{M}, \mathbf{u})$.
- Receive($\text{cor}_R = K'$, DB, $\mathbf{c} = ((S'_\ell, Z_\ell, i_\ell)_{\ell \in [\lambda]}, \mathbf{M}, \mathbf{u})$) :
- For all $\ell \in [\lambda]$, parse $Z_\ell = (\mathbf{z}_{\ell,j})_{j \in S'_\ell}$.
 - Compute and output

$$\mathbf{y} \leftarrow \mathbf{u} - \sum_{\ell \in [\lambda]} \left(\mathbf{M} \cdot \text{PRF}'_{K'}(\ell, i_\ell) + \sum_{j \in S'_\ell} \mathbf{z}_{\ell,j} \text{DB}[j] \right).$$

4.2 Efficiency

We will briefly analyze the computational and communication efficiency of the scheme `plvOLE`. When instantiating the covering set generator with the construction of [21] discussed in Section 2.4, we obtain the following efficiency parameters. The important parameters in this context are the security parameter and the database-size N , which we treat as an independent parameter as we think of N to be a large polynomial in λ . Recall that for this instantiation we have $n = \sqrt{N}$, $m = \lambda\sqrt{N}$ and $\delta \approx 1/\sqrt{N}$. Recall further that in this instantiation the seeds seed are of size $(n + m) \log(N) = O(\lambda \log(N) \sqrt{N})$

- The receiver preprocessing $\text{cor}_R = K'$ is a single PRF key of size λ , independent of N . Hence $|\text{cor}_R| = \lambda$.
- The sender preprocessing $\text{cor}_S = (K, (\text{seed}_\ell, (\mathbf{h}_{\ell,i})_{i \in [m]})_{\ell \in [\lambda]})$ consists of a PRF key K of size λ and λ of the following: A seed seed_ℓ of size $O(\lambda \log(N) \sqrt{N})$ and $m = \lambda\sqrt{N}$ hints $\mathbf{h}_{\ell,i}$, each of size $(k + \lambda) \log(|\mathbb{F}|)$. Hence, the total size of cor_S comes down to

$$\begin{aligned} |\text{cor}_S| &= O(\lambda^2 \cdot (\log(N) + (k + \lambda) \log(|\mathbb{F}|)) \sqrt{N}) \\ &= O(\lambda^2 \cdot (k + \lambda) \log(|\mathbb{F}|) \sqrt{N}). \end{aligned}$$

- The sender message $\mathbf{c} = ((S'_\ell, Z_\ell, i_\ell)_{\ell \in [\lambda]}, \mathbf{M}, \mathbf{u})$ consists of a matrix $\mathbf{M}$ of size $k(k + \lambda) \log |\mathbb{F}|$, a vector $\mathbf{u}$ of size $k \cdot \log |\mathbb{F}|$ and λ of the following: A set S'_ℓ of bit-size $n \cdot \log(N) = \log(N) \cdot \sqrt{N}$, a list of vectors $Z_\ell = (\mathbf{z}_{\ell,j})_{j \in S'_\ell}$ of bit-size $\log(N) \cdot \sqrt{N} \cdot k \cdot \log(|\mathbb{F}|)$ and an index i_ℓ of size $\log(m)$. The total size comes down to

$$|\mathbf{c}| = O(k(k + \lambda))(\log(|\mathbb{F}|) + \log(N)\sqrt{N} \cdot k \cdot \log(|\mathbb{F}|)).$$

- The computation of the **Send** algorithm is dominated by the overhead of **SampleSet**, which is $\text{poly}(\lambda) \cdot \sqrt{N}$. Computing the matrix $\mathbf{M}$ has overhead $\text{poly}(k, \lambda) = \text{poly}(\lambda)$, whereas computing the lists Z_ℓ has overhead $n \cdot \text{poly}(k, \lambda) = \text{poly}(\lambda) \cdot \sqrt{N}$.
- The computation of the **Receive** algorithm is dominated by computing the sum, and hence comes down to $\text{poly}(\lambda)\sqrt{N}$.

4.3 Correctness

We will first establish correctness of our protocol. If PRF is “only” a pseudorandom function, correctness of our protocol holds only computationally, as the sender needs to choose a suitable matrix $\mathbf{M}$, whose existence depends on certain constraints, which hold for uniformly random values. However, if PRF_K is additionally a λ -wise uniform function, correctness holds statistically. Note that any pseudorandom function PRF can be made t -wise independent by xor-ing the output of a t -wise independent function on the output of PRF. For simplicity, we will prove the following Lemma for the case that PRF is a λ -wise uniform function.

Lemma 10. *Assume that PRF is a λ -wise independent function. Then the protocol plvOLE satisfies the correctness property.*

Proof. First observe that given that $\text{Send}(\text{cor}_S, x, \mathbf{v}, \mathbf{w})$ does not output $\perp$, it holds that

$$\mathbf{y} = \mathbf{u} - \sum_{\ell \in [\lambda]} \left(\mathbf{M} \cdot \text{PRF}'_{K'}(\ell, i_\ell) + \sum_{j \in S'_\ell} \mathbf{z}_{\ell,j} \text{DB}[j] \right).$$

We can expand $\mathbf{u}$ as

$$\begin{aligned} \mathbf{u} &= \mathbf{w} + \mathbf{M} \cdot \left(\sum_{\ell \in [\lambda]} \mathbf{h}_{\ell, i_\ell} \right) \\ &= \mathbf{w} + \mathbf{M} \cdot \left(\sum_{\ell \in [\lambda]} \left(\text{PRF}'_{K'}(\ell, i_\ell) + \sum_{j \in S_{\ell, i_\ell}} \text{PRF}_K(\ell, i_\ell, j) \cdot \text{DB}[j] \right) \right) \end{aligned}$$

Hence it holds that

$$\begin{aligned} \mathbf{y} &= \mathbf{w} + \mathbf{M} \cdot \left(\sum_{\ell \in [\lambda]} \left(\sum_{j \in S_{\ell, i_{\ell}}} \text{PRF}_K(\ell, i_{\ell}, j) \cdot \text{DB}[j] - \sum_{j \in S'_{\ell}} \text{PRF}_K(\ell, i_{\ell}, j) \text{DB}[j] \right) \right) \\ &= \mathbf{w} + \mathbf{M} \cdot \left(\sum_{\ell \in [\lambda]} \text{PRF}_K(\ell, i_{\ell}, x'_{\ell}) \cdot \text{DB}[x'_{\ell}] \right) \end{aligned} \quad (1)$$

$$\begin{aligned} &= \mathbf{w} + \mathbf{M} \cdot \left(\sum_{\ell \in L} \text{PRF}_K(\ell, i_{\ell}, x'_{\ell}) \cdot \text{DB}[x'_{\ell}] \right) + \sum_{\ell \in [\lambda] \setminus L} \underbrace{\mathbf{M} \cdot \text{PRF}_K(\ell, i_{\ell}, x'_{\ell}) \cdot \text{DB}[x'_{\ell}]}_{=0} \\ &= \mathbf{w} + \mathbf{M} \cdot \left(\sum_{\ell \in L} \text{PRF}_K(\ell, i_{\ell}, x) \right) \cdot \text{DB}[x] \end{aligned} \quad (2)$$

$$= \mathbf{w} + \mathbf{v} \cdot \text{DB}[x] \quad (3)$$

Equality (1) follows by correctness of `SetGen`, i.e. it holds that $S_{\ell, i_{\ell}} = S'_{\ell} \cup \{x'_{\ell}\}$. Furthermore, equality (2) follows from the fact that for all $\ell \in L$ it holds that $x'_{\ell} = x$ (this is in fact the definition of L) and the fact that for all $\ell \notin L$ it holds that $\mathbf{M} \cdot \text{PRF}_K(\ell, i_{\ell}, x'_{\ell})$ (this is how $\mathbf{M}$) was constructed. Likewise, equality (3) also follows from the way we defined $\mathbf{M}$.

Hence, it remains to bound the probability that `Send`(`corS`, x , $\mathbf{v}$, $\mathbf{w}$) outputs $\perp$.

By a Chernoff-bound it holds that $|L| > \lambda/2$, except with negligible probability. Hence assume in the following that L is non-empty. Observe that if the vectors $\text{PRF}_K(\ell, i_{\ell}, x'_{\ell})$ for $\ell \in [\lambda] \setminus L$ and $\sum_{\ell \in L} \text{PRF}_K(\ell, i_{\ell}, x)$ are linearly independent, then by Lemma 3 we can always find a full-rank matrix $\mathbf{M}$ with $\mathbf{M} \cdot \text{PRF}_K(\ell, i_{\ell}, x'_{\ell}) = 0$ for $\ell \in [\lambda] \setminus L$ and $\mathbf{M} \cdot (\sum_{\ell \in L} \text{PRF}_K(\ell, i_{\ell}, x)) = \mathbf{v}$. Note that such an $\mathbf{M}$ can be found using the basic gaussian elimination algorithm.

Observe that the $\ell \in [\lambda] \setminus L$ and (say) the any fixed $\ell_0 \in L$ are pairwise different. Hence, as `PRF` is λ -wise independent, it holds that that vectors $\text{PRF}_K(\ell, i_{\ell}, x'_{\ell})$ for $\ell \in [\lambda] \setminus L$ and $\sum_{\ell \in L} \text{PRF}_K(\ell, i_{\ell}, x)$ are distributed independently uniformly random over $\mathbb{F}^{k+\lambda}$. Hence, by Lemma 2 these vectors are linearly independent, except with probability $O(\lambda \cdot q^{-k-\lambda/2-1})$, which is negligible. We conclude that the probability that `Send` outputs $\perp$ is negligible, which concludes the proof.

4.4 Receiver Privacy

Receiver privacy of `plvOLE` follows routinely from the pseudorandomness of `PRF'`.

Lemma 11. *Assume that `PRF'` satisfies the pseudorandomness property. Then the protocol `plvOLE` has receiver privacy.*

Proof. Observe that in the sender preprocessing $\text{cor}_S \leftarrow (\text{seed}_{\ell}, K, (\mathbf{h}_{\ell, i})_{i \in [m]})_{\ell \in [\lambda]}$ only the hints $\mathbf{h}_{\ell, i} = \text{PRF}'_{K'}(\ell, i) + \sum_{j \in S_{\ell, i}} \text{PRF}_K(\ell, i, j) \cdot \text{DB}[j]$ depend on the

database DB. By appealing to the pseudorandomness property of PRF' , we can replace the terms $\text{PRF}'_{K'}(\ell, i)$ with uniformly random values. After this transformation, the hints $h_{\ell, i}$ are independently uniform in the view of the sender. Hence receiver privacy follows.

4.5 Sender Privacy

We will now establish security of our protocol plvOLE given above.

Theorem 12. *Assume that PRF is a pseudorandom function and that $\text{SetGen} = (\text{Gen}, \text{SampleSet})$ satisfies the hiding property. Then the protocol plvOLE satisfies sender privacy.*

Proof. We will start by providing a simulator $\text{Sim}_R(\text{DB}, \mathbf{y})$, which proceeds as follows

- Choose $K' \leftarrow \{0, 1\}^\lambda$ uniformly random.
- Choose sets $S'_1, \dots, S'_\lambda \subseteq [N]$ and indices $i_1, \dots, i_\lambda \in [m]$ via $(S'_\ell, i_\ell) \leftarrow \text{Sim}_{\text{SetGen}}(1^\lambda)$ for all $\ell \in [\lambda]$.
- For $\ell \in [\lambda]$ and $j \in S'_\ell$ choose $\mathbf{z}_{\ell, j} \leftarrow_{\$} \mathbb{F}^k$ uniformly at random. For $\ell \in [\lambda]$ set $Z_\ell = (\mathbf{z}_{\ell, j})_{j \in S'_\ell}$.
- Compute $\mathbf{u} \in \mathbb{F}^k$ via $\mathbf{u} \leftarrow \mathbf{y} + \sum_{\ell \in [\lambda]} \left(\mathbf{M} \cdot \text{PRF}'_{K'}(\ell, i_\ell) + \sum_{j \in S'_\ell} \mathbf{z}_{\ell, j} \cdot \text{DB}[j] \right)$.
- Output $\text{cor}_R = K'$ and $\mathbf{c} = ((S'_\ell, Z_\ell, i_\ell)_{\ell \in [\lambda]}, \mathbf{M}, \mathbf{u})$.

We will now show that for any PPT-distinguisher $\mathcal{D}$, the sender-messages c produced by the real protocol are indistinguishable from c produced by the simulator Sim . Our proof will proceed in the following hybrids.

- $\mathcal{H}_0$: This is the real experiment.
- $\mathcal{H}_1$: This experiment is identical $\mathcal{H}_0$, however we compute $\mathbf{u}$ via

$$\mathbf{u} \leftarrow \mathbf{y} + \sum_{\ell \in [\lambda]} \left(\mathbf{M} \cdot \text{PRF}'_{K'}(\ell, i_\ell) + \sum_{j \in S'_\ell} \mathbf{z}_{\ell, j} \cdot \text{DB}[j] \right).$$

Statistical indistinguishability follows from the correctness of the scheme.

- $\mathcal{H}_2$: This experiment is identical to $\mathcal{H}_1$, but we replace all PRF outputs $\text{PRF}_K(\ell, i, j)$ with truly random values $\mathbf{r}_{\ell, i, j}$. In particular, we choose $\mathbf{M}$ as a uniformly random matrix satisfying $\mathbf{M} \cdot (\sum_{\ell \in L} \mathbf{r}_{\ell, i_\ell, x'_\ell}) = \mathbf{v}$ and $\mathbf{M} \cdot \mathbf{r}_{\ell, i_\ell, x'_\ell} = \mathbf{0}$ for all $\ell \in [\lambda] \setminus L$. Furthermore we compute the $\mathbf{z}_{\ell, j}$ via $\mathbf{z}_{\ell, j} = \mathbf{M} \cdot \mathbf{r}_{\ell, i_\ell, j}$. Computational indistinguishability of hybrids $\mathcal{H}_0$ and $\mathcal{H}_1$ follows routinely from the pseudorandomness property of PRF.
- $\mathcal{H}_3$: This experiment is identical to $\mathcal{H}_2$, but instead of computing the $\mathbf{z}_{\ell, j}$ via $\mathbf{z}_{\ell, j} = \mathbf{M} \cdot \mathbf{r}_{\ell, i_\ell, j}$, we choose the $\mathbf{z}_{\ell, j}$ uniformly random from $\mathbb{F}^k$. Hybrids $\mathcal{H}_2$ and $\mathcal{H}_3$ are identically distributed, as $\mathbf{M}$ has full rank and the $\mathbf{r}_{\ell, i_\ell, j}$ are independently uniform. Therefore $\mathbf{z}_{\ell, j} = \mathbf{M} \cdot \mathbf{r}_{\ell, i_\ell, j}$ also follows the uniform distribution over $\mathbb{F}^k$.

- $\mathcal{H}_4$: This experiment is identical to $\mathcal{H}_3$, except that we choose $\mathbf{M} \in \mathbb{F}^{k \times (k+\lambda)}$ as a uniformly random full-rank matrix. First observe that the $\mathbf{r}_{\ell, i_\ell, x'_\ell}$ for $\ell \in L$ and the $\mathbf{r}_{\ell, i_\ell, x'_\ell}$ for $\ell \in [\lambda] \setminus L$ are only used to sample $\mathbf{M}$ in $\mathcal{H}_3$, but nowhere else. Furthermore, as the $\mathbf{r}_{\ell, i_\ell, x'_\ell}$ for $\ell \in L$ are uniformly random, so is $\sum_{\ell \in L} \mathbf{r}_{\ell, i_\ell, x'_\ell}$. Consequently, we can invoke Lemma 3 with $\mathbf{b}_1 = \mathbf{v}$ and $\mathbf{b}_j = 0$ and conclude that $\mathbf{M}$ follows a uniform distribution. Consequently, hybrids $\mathcal{H}_3$ and $\mathcal{H}_4$ are identically distributed.
- $\mathcal{H}_5$: This hybrid is identical to $\mathcal{H}_4$, except that we compute the S'_ℓ and x_ℓ via $(S'_\ell, x_\ell) \leftarrow \text{Sim}_{\text{SetGen}}(1^\lambda)$ for all $\ell \in [\lambda]$. Indistinguishability follows routinely from the hiding property of SetGen . In this experiment, the view of the receiver is identically distributed to the output of the simulator Sim_R .

This concludes the proof.

5 Laconic Private Set Intersection with Preprocessing

In the laconic private set intersection protocol (PSI), a receiver possessing a potentially large set X of size m and a sender with a smaller set Y of size n compute the intersection $X \cap Y$ while ensuring that no additional information about their respective sets is disclosed. Previous works have required linear computational complexity on the receiver's end. In contrast, we introduce a novel private laconic vector OLE with preprocessing to develop a laconic PSI protocol in the preprocessing model, in which the receiver's computation during the online phase is sublinear. We introduce our laconic PSI construction in Construction 2, while the corresponding ideal functionality is provided in Figure 1.

Basic Idea: Our protocol constructs a laconic private set intersection (PSI) scheme leveraging the private laconic vector-OLE and universal hashing. Given the sender's input y , the receiver first constructs a database DB where each element $x \in X$ is assigned to a bin such that $\text{DB}[h(x)] = x$. Subsequently, the sender and receiver execute a private laconic vector-OLE, where the sender's inputs are defined as $v = -r$ and $w = -(r+1)y$ and index $i = h(y)$ where r is chosen randomly. Meanwhile, the receiver's input is DB . The OLE is then computed as:

$$u = v \cdot \text{DB}[i] + w$$

which evaluates to y if $\text{DB}[i] = y$; otherwise, it appears as a random value,

$$u = r(\text{DB}[i] - y) + y$$

from the receiver's perspective. Finally, the receiver outputs u if it belongs to its input set; otherwise, it outputs $\perp$.

Handling the collisions. Since we use universal hashing and set the database size to $c(\lambda)$ times the input set size m , collisions will inevitably occur during database construction. To address this, we employ multiple hash functions to generate multiple variants of the database. This approach allows us to repeat the OLE protocol, ensuring that each element $x \in X$ appears in at least one of the databases. If any OLE instance produces a valid output z (i.e., $z \in X$), we return z ; otherwise, we output $\perp$. In the correctness analysis, we demonstrate that, by using a sufficient number of hash functions, each $x \in X$ will be included in at least one database with very high probability.

Extending Sender set Y for $|Y| \geq 2$. In our protocol, we assume that the sender possesses a single element y . This assumption is justified by the fact that a larger set can be processed element-wise by executing the protocol separately for each element. However, to mitigate information leakage, the sender's set must be randomly permuted prior to execution. Without this precaution, the order of the elements could inadvertently reveal sensitive information. For example, if the set were sorted and the second element was found in the intersection, this would reveal that the first element is smaller than a specific value, namely, the second element, thereby resulting in unintended information leakage.

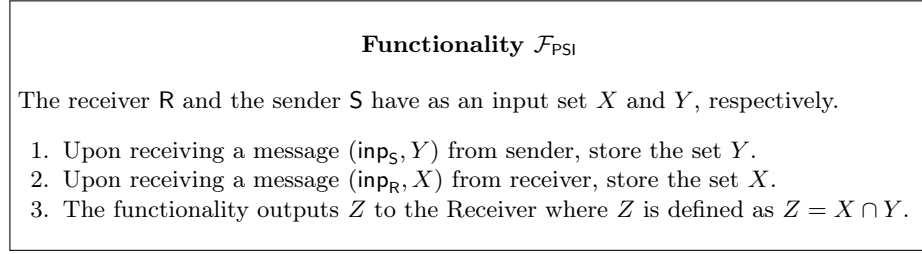


Fig. 1: PSI Functionality.

Construction 2 (Laconic PSI with Preprocessing) Let $c = c(\lambda) = \text{poly}(\lambda)$, X be an input set of the receiver with size m where each element is represented by ℓ bits, let y be an input element of the sender and $\mathbb{F}$ be a field of size at least $2^{\ell+\lambda+O(\log(\lambda))}$. Define φ as a bijective function: $\{0, 1\}^{\ell+\lambda+O(\log(\lambda))} \rightarrow \mathbb{F}$, which acts as a mapping from string bits to a field. Let $\mathcal{H}$ be a family of universal hash functions: $\{0, 1\}^\ell \mapsto [c \cdot m]$ and $\text{plvOLE} = (\text{Offline}, \text{Send}, \text{Receive})$ be a private laconic Vector-OLE with preprocessing. Then the scheme $\text{LPSI} = (\text{LPSI.Offline}, \text{LPSI.Send}, \text{LPSI.Receive})$ is given as follows:

- $(\text{cor}_S, \text{cor}_R) \leftarrow \text{LPSI.Offline}(X)$:
 1. Sample $h_1, \dots, h_t \leftarrow \mathcal{H}$.
 2. For every $i \in [t]$:
 - (a) Initialize a database DB_i of length $c \cdot m$ such that $\text{DB}_i[j] = 0$ for all $j \in [n]$. For every $x \in X$, set $\text{DB}_i[h_i(x)] = \varphi(x || 0^{\lambda+O(\log \lambda)})$.

- (b) Compute $\text{cor}_{S_i}, \text{cor}_{R_i} \leftarrow \text{plvOLE.Offline}(1^\lambda, 1, \text{DB}_i)$.
- 3. Set $\text{cor}_S = ((h_i)_{i \in [t]}, (\text{cor}_{S_i})_{i \in [t]})$, $\text{cor}_R = ((h_i)_{i \in [t]}, (\text{cor}_{R_i})_{i \in [t]})$ and output $(\text{cor}_S, \text{cor}_R)$.
- $\text{msg} \leftarrow \text{LPSI.Send}(\text{cor}_S = ((h_i)_{i \in [t]}, (\text{cor}_i)_{i \in [t]}), y)$:
 - 1. For every $i \in [t]$:
 - (a) $r_i \leftarrow \$ \mathbb{F}$
 - (b) Set $v_i = -r_i$ and $w_i = (r_i + 1) \cdot \varphi(y || 0^{\lambda+O(\log \lambda)})$
 - 2. For every $i \in [t]$, compute $e_i \leftarrow \text{plvOLE.Send}(\text{cor}_i, h_i(y), v_i, w_i)$
 - 3. Set $\text{msg} = (e_i)_{i \in [t]}$.
- $c \leftarrow \text{LPSI.Receive}(\text{cor}_R = ((h_i)_{i \in [t]}, (\text{cor}_i)_{i \in [t]}), X, \text{msg} = (e_i)_{i \in [t]})$:
 - 1. For every $i \in [t]$:
 - (a) Initialize a database DB_i of length $c \cdot \ell$ such that $\text{DB}_i[j] = 0$ for all $j \in [n]$. For every $x \in X$, set $\text{DB}_i[h_i(x)] = \varphi(x || 0^{\lambda+O(\log \lambda)})$.
 - (b) Compute $z_i = \text{plvOLE.Receive}(\text{cor}_i, \text{DB}_i, e_i)$
 - (c) If $\varphi^{-1}(z_i) = (z || 0^{\lambda+O(\log \lambda)})$ then output z .
 - 2. If none of the z_i were well-formed output $\perp$.

5.1 Correctness

In this section, we establish the correctness of the laconic PSI protocol. To establish correctness, we analyze two potential cases where the correctness fails. First, an error may occur if an element $x \in X$ is not stored in any of the t databases due to collisions from the hash functions. Second, an error may arise if an element $y \notin X \cap y$ is mistakenly identified as part of the intersection due to randomness in the protocol, leading to an incorrect output $z' \neq y$. We show that the probability of either of these failure events occurring is negligible.

Focusing on the first error, define a collision between two elements x, x' under a hash function h as occurring when $h(x) = h(x')$. First, we demonstrate that for every element $x \in X$, there exists at least one hash function h_i such that x does not collide with any other input element with very high probability. An alternative way to conceptualize this is that, in at least one of the t databases DB_i , the element x is correctly stored. Specifically, this means that

$$\text{DB}_i[h_i(x)] = \varphi(x || 0^{\lambda+O(\log \lambda)})$$

where φ represents an encoding function that maps elements from $\{0, 1\}^{\ell+\lambda+O(\log \lambda)}$ to $\mathbb{F}$

Lemma 13. *Let $h_1, \dots, h_t$ be hash functions randomly and independently sampled from a family of hash functions $\mathcal{H}$. Then for each $x \in X$, there exists an $i \in [t]$ such that $\text{DB}_i[h_i(x)] = \varphi(x || 0^{\lambda+O(\log \lambda)})$ except with probability $m \cdot \left(\frac{1}{c}\right)^t$ where $m = |X|$.*

Proof. By the definition of universal hash functions (see Definition 6), for every $x \in X$ and $i \in [t]$, the probability that x collides with another element $x' \in X$ under h_i is given by:

$$\Pr [h_i(x) = h_i(x') : x' \in X] \leq \frac{1}{cm}$$

By applying the union-bound for all $x' \in X/\{x\}$, we obtain:

$$\begin{aligned} \Pr [h_i(x) = h_i(x') : \exists x' \in X] &\leq \sum_{j \in [m-1]} \Pr [h_i(x) = h_i(x'_j) : x_j \in X/\{x\}] \\ &\leq \sum_{j \in [m-1]} \frac{1}{cm} \\ &\leq \frac{m-1}{cm} \\ &\leq \frac{1}{c} \end{aligned}$$

For every $x \in X$ to experience a collision in all hash functions h_i , the probability is bounded as follows:

$$\begin{aligned} \Pr [h_i(x) = h_i(x') : \forall i \in [t], \exists x' \in X/\{x\}] &\leq \prod_{i \in [t]} \Pr [h_i(x) = h_i(x') : \exists x' \in X] \\ &\leq \prod_{i \in [t]} \frac{1}{c} \\ &\leq \left(\frac{1}{c}\right)^t \end{aligned}$$

Finally, applying the union bound over all $x \in X$, we get:

$$\begin{aligned} \Pr [h_i(x) = h_i(x') : \forall x \in X, \forall i \in [t], \exists x' \in X/\{x\}] &\leq \sum_{x \in X} \Pr [h_i(x) = h_i(x') : \forall i \in [t], \exists x' \in X/\{x\}] \\ &\leq \sum_{x \in X} \left(\frac{1}{c}\right)^t \\ &\leq m \cdot \left(\frac{1}{c}\right)^t \end{aligned}$$

Thus, with probability at least $1 - m \cdot (1/c)^t$, there exists at least one $i \in [t]$ such that $h_i(x)$ does not collide with any other element in X , ensuring that x is correctly stored in DB_i .

We now analyze the case where there exists a database DB_i such that an element x is correctly stored in the database, i.e., there exists an index $j \in [c \cdot m]$ such that:

$$\text{DB}_i[j] = \varphi(x || 0^{\lambda + O(\log \lambda)})$$

If the sender's input element y belongs to X , the correctness of **plvOLE** ensures that at least one instance of **plvOLE** outputs y . Specifically, if y is present in DB_i , i.e.,

$$\text{DB}_i[j] = \varphi(y || 0^{\lambda+O(\log \lambda)})$$

then the i^{th} instance of **plvOLE** outputs y . In contrast, if $y \notin X$, then either y is hashed to an empty entry in the database,

$$\text{DB}_i[h_i(y)] = 0$$

or a collision occurs such that

$$\text{DB}_i[j] = \varphi(x' || 0^{\lambda+O(\log \lambda)})$$

for some $x' \in X$. In both cases, the output of the i^{th} **plvOLE** instance will be a random value due to the randomness of r_i . Mathematically, in the absence of a collision, the i^{th} **plvOLE** instance outputs

$$(r_i + 1)\varphi(y || 0^{\lambda+O(\log \lambda)})$$

In the case of a collision, the output is given by

$$r_i(\varphi(y || 0^{\lambda+O(\log \lambda)}) - \varphi(x' || 0^{\lambda+O(\log \lambda)})) + \varphi(y || 0^{\lambda+O(\log \lambda)})$$

The receiver will output z_i only if, based on the random choice of r_i , the output of **plvOLE** is of the form $z || 0^{\lambda+O(\log \lambda)} = \varphi(z_i)$.

This event occurs with probability at most $\frac{1}{2^{\lambda+O(\log \lambda)}}$. Since the protocol executes t instances of **plvOLE**, the probability of producing an incorrect output is bounded by $\frac{t}{2^{\lambda+O(\log \lambda)}}$ by applying the union bound across all **plvOLE** instances. We now combine the two failure probabilities using a union bound. The two sources of error are the following.

- **Collisions while constructing the t databases:** This occurs with probability at most

$$m \cdot \left(\frac{1}{c}\right)^t$$

- **Incorrect output generation:** This occurs with probability at most

$$\frac{t}{2^{\lambda+O(\log \lambda)}}$$

Thus, the total error probability is given by

$$m \cdot \left(\frac{1}{c}\right)^t + \frac{t}{2^{\lambda+O(\log \lambda)}}$$

This probability can be made negligible by choosing $t = O(\lambda)$. Therefore, we conclude that the correctness of the protocol holds except with negligible probability, completing the proof.

5.2 Efficiency

We briefly analyze the computational and communication efficiency of the laconic PSI protocol. We set the parameters as $N = c \cdot m$, $k = 1$ and $t = O(\lambda)$.

- The sender preprocessing cor_S consists of t hash functions which can be represented by $O(\lambda \cdot \ell)$ bits and t plvOLE sender's preprocessing of size $O(\lambda^3 \log(|\mathbb{F}|) \sqrt{cm})$. Thus, the total size of the sender's preprocessing is $O(\lambda^2 \ell + \lambda^4 \log(|\mathbb{F}|) \sqrt{cm})$.
- The receiver preprocessing cor_R consists of t hash functions of size $O(\lambda \ell)$ and t plvOLE receiver's preprocessing of size λ totaling of size $O(\lambda^2 \ell)$.
- The **Send** message consist of t instances of send message of plvOLE which equals to $O(\lambda^2)(\log(|\mathbb{F}|) + \log(cm) \sqrt{cm} \cdot \log(|\mathbb{F}|))$
- The running time of the **Send** and **Receive** algorithm is dominated by t instances of plvOLE, which results in a running time of $\text{poly}(\lambda) \sqrt{cm}$.

This analysis highlights that our laconic PSI protocol achieves sublinear computational complexity for the sender and receiver while maintaining efficient communication overhead.

5.3 Security

Theorem 1 *Let plvOLE be a private laconic vector OLE with preprocessing and $\mathcal{H}$ be a family of universal hash functions: $\{0, 1\}^\ell \mapsto [c \cdot m]$. Then, the laconic PSI with preprocessing protocol as shown in Construction 2 securely implements $\mathcal{F}_{\text{PSI}}$ (Figure 1) in the semi-honest model.*

Proof. Intuitively, the security against a corrupted sender follows from the receiver's privacy of the plvOLE, i.e., the correlations do not leak anything against the input set of the receiver. The security against a corrupted receiver can be split into two cases - one where the output is a value z and the other case where the output is $\perp$. If the output is z , we reduce security to the sender's privacy of the plvOLE, running the simulator of plvOLE such that z is the output. To simulate the hashing collisions, we use z with probability $1 - \frac{1}{c}$ and r_i otherwise. Finally, we consider the case where z is the output but not in the intersection. As explored in the correctness argument, this happens with probability $\frac{t}{2\lambda}$. In the case where $\perp$ is the output, we invoke the simulator of plvOLE with random values as the output of the receiver, as in the real execution, plvOLE outputs a random value from the view of the receiver (a similar argument is made in the correctness argument).

Security against a Semi-Honest Sender. The simulator Sim takes in input $1^\lambda, cm, t$ and runs t instances of simulator Sim_S of plvOLE (defined in Definition 9 under receiver privacy) and outputs these t simulated views returned by Sim_S . Using a standard hybrid argument, it can be shown that Sim 's output is indistinguishable from the real execution via a reduction to the receiver's privacy of plvOLE.

Security against a Semi-Honest Receiver. On input, the parameters 1^λ , cm , t , and output z , we simulate the view of a corrupted receiver in the following way:

1. Sample $h_1, \dots, h_t \leftarrow \$ \mathcal{H}$.
2. If $z = \perp$, then for each $i \in [t]$, compute $(\text{cor}_{R_i}, \text{msg}_i) = \text{Sim}_R(1^\lambda, cm, r_i)$ for $r_i \leftarrow \$ \mathbb{F}$.
3. Otherwise, for each $i \in [t]$, compute $(\text{cor}_{R_i}, \text{msg}_i) = \text{Sim}_R(1^\lambda, cm, r_i)$ where r_i is chosen randomly with probability $\frac{1}{c}$ or set to $\varphi(y||0^\lambda)$ with probability $1 - \frac{1}{c}$.
4. Set $\text{cor}_R = (\text{cor}_{R_i})_{i \in [t]}$, $\text{msg} = (\text{msg}_i)_{i \in [t]}$ and output $(\text{cor}_R, \text{msg})$.

Where Sim_R is the simulator of plvOLE for sender privacy against a semi-honest receiver.

We next show that:

$$(\text{cor}_R, \text{Send}(\text{cor}_S, y)) \approx_c \text{Sim}(1^\lambda, cm, t, z)$$

This analysis is split into two cases:

Case 1: When $z = \perp$, then we show that the simulated view is indistinguishable from the view in the real execution using the following sequence of hybrids:

- $\mathcal{H}_0$: Identical to the receiver's view in the real protocol.
- For all $i \in [t]$, replace all inputs of the i^{th} plvOLE with random values by setting a_i , b_i and $h_i(y)$ randomly. $\mathcal{H}_0$ and $\mathcal{H}_1$ are computationally indistinguishable because if $y \notin X$, then plvOLE outputs a random value and thereby can replace all inputs with random values. In the case that there is a collision while constructing DB_i and $y \in X$, then the protocol only outputs $\perp$ if $\forall i \in [t] \text{DB}_i[h_i(y)] = \varphi(x'||0^\lambda)$. This also results in plvOLE outputting a random value, and thereby, all inputs can be replaced with random values.
- $\mathcal{H}_2$: Replace each plvOLE with the Simulator of plvOLE . $\mathcal{H}_1$ and $\mathcal{H}_2$ are computationally indistinguishable, which directly follows from the sender privacy of plvOLE . This hybrid is identical to the receiver's view in the simulated protocol.

Case 2: When $z \neq \perp$, then we show that the simulated view is indistinguishable from the view in the real execution using the following sequence of hybrids:

- $\mathcal{H}_0$: Identical to the receiver's view in the real protocol.
- $\mathcal{H}_1$: For all $i \in [t]$ where i^{th} instance of plvOLE doesn't output z , replace each input of plvOLE such as a_i , b_i and $h_i(y)$ with random values. For plvOLE that do not output a valid value or z , then $\text{DB}_i[h_i(y)] = \varphi(x'||0^\lambda)$ and outputs a random value from the receiver's view. Therefore, random values can be replaced, and $\mathcal{H}_0$ and $\mathcal{H}_1$ are computationally indistinguishable.
- $\mathcal{H}_2$: Replace each plvOLE with the Simulator of plvOLE with the plvOLE outputting z will have the simulator of plvOLE input z while other Simulators input random values. $\mathcal{H}_1$ and $\mathcal{H}_2$ are computationally indistinguishable and follows from the sender privacy of plvOLE .

- $\mathcal{H}_3$: Permute all the simulators of plvOLE . $\mathcal{H}_2$ and $\mathcal{H}_3$ are identically distributed.
- $\mathcal{H}_4$: For each simulator of plvOLE , input z with probability $1 - \frac{1}{c}$ and input a random value with probability $\frac{1}{c}$. $\mathcal{H}_3$ and $\mathcal{H}_4$ are computationally indistinguishable because the inputs are chosen based on the probability of collision in the database DB_i . With probability $\frac{1}{c}$, there is a collision in the database for z , and, due to this collision, the output of plvOLE or the simulator would be random. Therefore, the distribution remains the same. This hybrid is identical to the receiver's view in the simulated protocol.

This concludes the proof.

6 Laconic Function Evaluation for RAM Programs

In this section, we discuss the main application of our private laconic vector OLE with preprocessing: laconic function evaluation for RAM programs (RAM-LFE) in our two-server model.

6.1 RAM Programs

We will briefly describe the RAM model considered in this paper. A RAM program P with runtime T as a sequence of step-circuits $C_1, \dots, C_T$. The circuits get access to a large input database and a work database, where the latter is initialized with zeros. Each circuit can make a single read query to the input database D , as well as a single read and a single write query to the work database W . More formally, each step-circuit C_i receives as input a state st_i , an input d which is the result of a read-query to D by C_{i-1} , and an input w which is the result of a write-query to W by C_{i-1} . C_i returns a state st_{i+1} , an index ind_D for a read-query on D and an index ind_W for a read-query on W , and a pair (ind, y) that instructs to write symbol y to location ind in database W .

We will make a few additional assumptions about the RAM program P , in particular, that the read-queries may fail. In this case, the read-query returns a special symbol $\perp$. We will briefly argue that if the probability of a read-query failing is sufficiently small, then a small increase in runtime can compensate for read errors without changing the overall program structure.

6.2 Construction

We will now present a construction of RAM-LFE for read-only RAM programs. We will then discuss how this can be extended to RAM-LFE for RAM programs that make both read and write requests via efficient garbled RAM schemes.

Definition 14 (RAM-LFE). *A laconic function evaluation for RAM programs (RAM-LFE) consists of the three algorithms (Offline, Send, Receive) with the following syntax, correctness, security and efficiency properties.*

- $\text{cor} \leftarrow \text{Offline}(1^\lambda, \text{DB}, T)$: Given the security parameter 1^λ , a database $\text{DB} \in \{0, 1\}^n$ and a runtime T , it outputs the correlation cor .
- $\text{e} \leftarrow \text{Send}(\text{cor}, P)$: Given the correlation cor , a program P , it outputs a ciphertext e .
- $m \leftarrow \text{Receive}(\text{DB}, \text{e})$: Given the database DB and a ciphertext e , it outputs a value m .

Correctness: For all $\lambda \in \mathbb{N}$, $n = \text{poly}(\lambda)$ for any polynomial function $\text{poly}(\cdot)$, any database $\text{DB} \in \{0, 1\}^n$ and any program P with runtime $T = \text{poly}(\lambda)$, it holds that

$$\Pr \left[m = P^{\text{DB}} \mid \begin{array}{l} \text{cor} \leftarrow \text{Offline}(1^\lambda, \text{DB}, T) \\ \text{e} \leftarrow \text{Send}(\text{cor}, P) \\ m \leftarrow \text{Receive}(\text{DB}, \text{e}) \end{array} \right] \geq \text{negl}(\lambda).$$

Sender Privacy Against Semi-honest Receivers: For all $\lambda \in \mathbb{N}$, $n = \text{poly}(\lambda)$ for any polynomial function $\text{poly}(\cdot)$, any database $\text{DB} \in \{0, 1\}^n$, any program P with runtime $T = \text{poly}(\lambda)$, it holds that there exists a PPT simulator Sim , such that

$$\text{Send}(\text{pp}, \text{cor}, P) \stackrel{\mathcal{C}}{\approx} \text{Sim}(\text{pp}, \text{DB}, m)$$

where $\text{cor} \leftarrow \text{Offline}(1^\lambda, \text{DB}, T)$.

Efficiency: The length of cor , and the runtime of the algorithm Offline , Send and Receive are sublinear in the size of the database n .

The idea for our read-only RAM-LFE follows naturally from combining private laconic VOLE correlations with garbled circuits: In the offline phase, we generate private laconic VOLE correlations with respect to the receiver's database DB for each step-circuit C_i , and in the online phase, we hardwire these correlations into an extended version C'_i of C_i to allow access to the database DB .

Hence, let C_i be a step circuit. Assume for convenience that C_i takes as input a state st_i as well as a bit w_i (which is the result of the previous read-query), and outputs a state st_{i+1} and an index ind for a read-query. The circuit C'_i has hardwired inputs $\text{lab}_{\text{st}_{i+1}}$, $\text{lab}_{w_{i+1}}$ and a private laconic VOLE correlation cor_i . Here, $\text{lab}_{\text{st}_{i+1}}$ are the input labels corresponding to st_{i+1} for a garbling of the next step-circuit C'_{i+1} , whereas $\text{lab}_{w_{i+1}}$ are likewise input labels corresponding to the bit w_{i+1} .

The augmented step circuit C'_i is given as follows.

- Circuit $\text{C}'_i[\text{pp}, \text{lab}_{\text{st}_{i+1}}, \text{lab}_{w_{i+1}}, \text{cor}_i](\text{st}_i, w_i)$:
 - Compute $(\text{st}_{i+1}, \text{ind}_{i+1}) \leftarrow \text{C}(\text{st}_i, w_i)$
 - Set $\text{lb}_{\text{st}_{i+1}} \leftarrow \text{LabelEncoding}(\text{lab}_{\text{st}_{i+1}}, \text{st}_{i+1})$
 - Compute $e_{i+1} \leftarrow \text{Send}(\text{cor}_i, x = \text{ind}_{i+1}, v = \text{lab}_{w_{i+1}, 1} - \text{lab}_{w_{i+1}, 0}, w = \text{lab}_{w_{i+1}, 0})$
 - Output $(\text{lb}_{\text{st}_{i+1}}, e_{i+1})$

Our RAM-LFE scheme $\text{RLFE} = (\text{RLFE.Offline}, \text{RLFE.Send}, \text{RLFE.Receive})$ in the preprocessing model is now given as follows.

Construction 3 (RAM-LFE) *Let $(\text{Offline}, \text{Send}, \text{Receive})$ be a private laconic VOLE with preprocessing scheme with sender privacy. Further $(\text{GC.Garble}, \text{GC.Eval})$ be a projective garbling scheme. The scheme $\text{RLFE} = (\text{RLFE.Setup}, \text{RLFE.Offline}, \text{RLFE.Send}, \text{RLFE.Receive})$ is given as follows.*

- $\text{RLFE.Offline}(1^\lambda, \text{DB}, T)$:
 1. For $i \in [T]$ compute $\text{cor}_i \leftarrow \text{Offline}(1^\lambda, \text{DB})$;
 2. Output $\text{cor}^* \leftarrow (\text{cor}_i)_{i \in [T]}$.
- $\text{RLFE.Send}(\text{cor}^*, P)$:
 1. Parse $\text{cor}^* = (\text{cor}_i)_{i \in [T]}$ and $P = (C_1, \dots, C_T)$;
 2. Compute $(\tilde{C}_T, \text{lb}_{\text{st}_T}, \text{lb}_{w_T}) \leftarrow \text{GC.Garble}(C_T)$;
 3. For $i = T-1, \dots, 1$ compute
 $(\tilde{C}_i, \text{lb}_{\text{st}_i}, \text{lb}_{w_i}) \leftarrow \text{GC.Garble}(C_i[\text{lb}_{\text{st}_{i+1}}, \text{lb}_{w_{i+1}}, \text{cor}_i])$;
 4. Set $\text{lb}_{\text{st}_1} = \text{LabelEncoding}(\text{lb}_{\text{st}_1}, 0^*)$, $\text{lb}_{w_1} = \text{LabelEncoding}(\text{lb}_{w_1}, 0)$;
 5. Output $e^* = (\text{lb}_{\text{st}_1}, \text{lb}_{w_1}, \tilde{C}_1, \dots, \tilde{C}_T)$.
- $\text{RLFE.Receive}(\text{DB}, e^*)$:
 1. Parse $e^* = (\text{lb}_{\text{st}_1}, \text{lb}_{w_1}, \tilde{C}_1, \dots, \tilde{C}_T)$;
 2. For $i = 1, \dots, T-1$:
 - (a) Compute $(\text{lb}_{\text{st}_{i+1}}, e_{i+1}) \leftarrow \text{GC.Eval}(\tilde{C}_i, \text{lb}_{\text{st}_i}, \text{lb}_{w_i})$
 - (b) Compute $\text{lb}_{w_{i+1}} \leftarrow \text{Receive}(\text{DB}, e_{i+1})$
 3. Compute and output $y \leftarrow \text{GC.Eval}(\tilde{C}_T, \text{lb}_{\text{st}_T}, \text{lb}_{w_T})$.

Correctness. The correctness of the above scheme RLFE routinely follows from the correctness of the underlying garbling scheme $\text{GC} = (\text{GC.Garble}, \text{GC.Eval})$ and the correctness of the private laconic VOLE with preprocessing scheme $(\text{Offline}, \text{Send}, \text{Receive})$.

Efficiency. The ciphertext size of RLFE is dominated by the hardwired correlations cor_i . In plVOLE , given that the size of cor_i is $O(\lambda^2 \cdot (k + \lambda) \log(|\mathbb{F}|) \sqrt{n})$, the total ciphertext size is $\text{poly}(\lambda) \cdot O(T\sqrt{n})$. Thus, due to the efficiency properties of the underlying private laconic VOLE with the preprocessing scheme, the overhead for the sender and receiver will also be $\text{poly}(\lambda) \cdot O(T\sqrt{n})$.

Sender Security. The following theorem establishes sender security for RLFE .

Theorem 2 *Assume that $\text{GC} = (\text{GC.Garble}, \text{GC.Eval})$ is a simulation secure garbling scheme and that $\text{plVOLE} = (\text{Offline}, \text{Send}, \text{Receive})$ has sender privacy against semi-honest receivers. Then the scheme $\text{RLFE} = (\text{RLFE.Offline}, \text{RLFE.Send}, \text{RLFE.Receive})$ also has sender privacy against semi-honest receivers.*

Proof. The proof proceeds via a hybrid argument. We will define $2T + 1$ hybrids $\mathcal{H}_0, \mathcal{H}_{i,0}$ and $\mathcal{H}_{i,1}$ for $i = 1, \dots, T$.

- $\mathcal{H}_0$: This is the real experiment.

- $\mathcal{H}_{i,0}$ is identical to $\mathcal{H}_{i-1,1}$ (or $\mathcal{H}_0$ if $i = 1$), except that we compute $\tilde{C}_i$, lb_{st_i} and lb_{w_i} via $(\tilde{C}_i, \text{lb}_{\text{st}_i}) \leftarrow \text{GC.Sim}(C_i[\text{pp}, \text{lab}_{\text{st}_{i+1}}, \text{lab}_{w_{i+1}}, \text{cor}_i](\text{st}_i, w_i))$. Indistinguishability follows by the simulation security of GC.
- $\mathcal{H}_{i,1}$ is identical to $\mathcal{H}_{i,0}$, except that we compute e_{i+1} (which is computed by C_i) via $e_{i+1} \leftarrow \text{Sim}(\text{lb}_{w_{i+1}}, \text{DB}_{\text{ind}_{i+1}})$. Indistinguishability follows by the sender privacy of laconic OT.

Note that in the last hybrid, the output of the receiver depends only on $C_T(\text{st}_T, w_T)$ (i.e., the correct output of P), but is otherwise independent of P , since all labels used during the simulation are chosen uniformly at random. This concludes the security proof. $\square$

Beyond read-only RAM Programs We will briefly discuss how the RLFE scheme can be upgraded to support RAM programs that support both read and write queries. Instead of relying on updatable laconic OT like previous work [20, 26], we can combine the scheme RLFE with a garbled RAM scheme [31, 33, 41, 47]. Specifically, such schemes also proceed by successively garbling the step circuits of the RAM, but also provide a garbled data-structure for the work database, which can be both read from and written to. For a garbled work database of size L , these constructions incur only an additive overhead of size $\tilde{O}(L)$ (in addition to the size of the garbled step circuits). Assuming that the work memory is initialized with zeros, we observe that a RAM program running in time T can read from and write to at most T locations of the work memory. Consequently, a work memory of size $O(T)$ is sufficient in this setting. We can obtain a full-fledged RAM LFE scheme by interleaving the circuits C'_i in our construction above (which essentially perform read queries on the receiver’s database) with the circuits C''_i originating from an underlying garbled RAM.

As a result, we get a fully-fledged RAM LFE scheme with ciphertext size, sender, and receiver overheads of $\text{poly}(\lambda) \cdot O(T \cdot \sqrt{n})$. By relying on a black-box garbled RAM scheme [31, 41] and storing the *expanded laconic OLE correlations*, the garbled circuits need only perform black-box operations. Consequently, for moderately large parameters n , the overhead of our scheme compares favorably to the RAM-LFE scheme in [26], which relies on heavy non-black-box techniques.

References

1. Alapati, N., Branco, P., Döttling, N., Garg, S., Hajiabadi, M., Pu, S.: Laconic private set intersection and applications. In: Nissim, K., Waters, B. (eds.) TCC 2021, Part III. LNCS, vol. 13044, pp. 94–125. Springer, Cham (Nov 2021)
2. Applebaum, B., Damgård, I., Ishai, Y., Nielsen, M., Zichron, L.: Secure arithmetic computation with constant computational overhead. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part I. LNCS, vol. 10401, pp. 223–254. Springer, Cham (Aug 2017)
3. Aranha, D.F., Lin, C., Orlandi, C., Simkin, M.: Laconic private set-intersection from pairings. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022. pp. 111–124. ACM Press (Nov 2022)

4. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: Feigenbaum, J. (ed.) CRYPTO'91. LNCS, vol. 576, pp. 420–432. Springer, Berlin, Heidelberg (Aug 1992)
5. Beaver, D.: Correlated pseudorandomness and the complexity of private computations. In: Miller, G.L. (ed.) Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22–24, 1996. pp. 479–488. ACM (1996)
6. Ben-Or, M., Goldwasser, S., Kilian, J., Wigderson, A.: Multi-prover interactive proofs: How to remove intractability assumptions. In: 20th ACM STOC. pp. 113–131. ACM Press (May 1988)
7. Bendlin, R., Damgård, I., Orlandi, C., Zakarias, S.: Semi-homomorphic encryption and multiparty computation. In: Paterson, K.G. (ed.) EUROCRYPT 2011. LNCS, vol. 6632, pp. 169–188. Springer, Berlin, Heidelberg (May 2011)
8. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y.: Compressing vector OLE. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 896–912. ACM Press (Oct 2018)
9. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Resch, N., Scholl, P.: Oblivious transfer with constant computational overhead. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part I. LNCS, vol. 14004, pp. 271–302. Springer, Cham (Apr 2023)
10. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Rindal, P., Scholl, P.: Efficient two-round OT extension and silent non-interactive secure computation. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019. pp. 291–308. ACM Press (Nov 2019)
11. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators: Silent OT extension and more. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019, Part III. LNCS, vol. 11694, pp. 489–518. Springer, Cham (Aug 2019)
12. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators from ring-LPN. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 387–416. Springer, Cham (Aug 2020)
13. Boyle, E., Gilboa, N., Ishai, Y.: Secure computation with preprocessing via function secret sharing. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019, Part I. LNCS, vol. 11891, pp. 341–371. Springer, Cham (Dec 2019)
14. Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) LWE. In: FOCS. pp. 97–106. IEEE Computer Society (2011)
15. Brakerski, Z., Vaikuntanathan, V.: Fully homomorphic encryption from ring-lwe and security for key dependent messages. In: CRYPTO. pp. 505–524 (2011)
16. Branco, P., Döttling, N., Mateus, P.: Two-round oblivious linear evaluation from learning with errors. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022, Part I. LNCS, vol. 13177, pp. 379–408. Springer, Cham (Mar 2022)
17. Brassard, G., Crépeau, C., Wolf, S.: Oblivious transfers and privacy amplification. *Journal of Cryptology* 16(4), 219–237 (Sep 2003)
18. Burra, S.S., Larraia, E., Nielsen, J.B., Nordholt, P.S., Orlandi, C., Orsini, E., Scholl, P., Smart, N.P.: High-performance multi-party computation for binary circuits based on oblivious transfer. *Journal of Cryptology* 34(3), 34 (Jul 2021)
19. Chang, Y.C., Lu, C.J.: Oblivious polynomial evaluation and oblivious neural learning. In: Boyd, C. (ed.) ASIACRYPT 2001. LNCS, vol. 2248, pp. 369–384. Springer, Berlin, Heidelberg (Dec 2001)

20. Cho, C., Döttling, N., Garg, S., Gupta, D., Miao, P., Polychroniadou, A.: Laconic oblivious transfer and its applications. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part II. LNCS, vol. 10402, pp. 33–65. Springer, Cham (Aug 2017)
21. Corrigan-Gibbs, H., Kogan, D.: Private information retrieval with sublinear online time. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part I. LNCS, vol. 12105, pp. 44–75. Springer, Cham (May 2020)
22. Couteau, G.: A note on the communication complexity of multiparty computation in the correlated randomness model. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part II. LNCS, vol. 11477, pp. 473–503. Springer, Cham (May 2019)
23. Damgård, I., Pastro, V., Smart, N.P., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 643–662. Springer, Berlin, Heidelberg (Aug 2012)
24. Damgård, I., Zakarias, S.: Constant-overhead secure computation of Boolean circuits using preprocessing. In: Sahai, A. (ed.) TCC 2013. LNCS, vol. 7785, pp. 621–641. Springer, Berlin, Heidelberg (Mar 2013)
25. Dong, F., Hao, Z., Mook, E., Wee, H., Wichs, D.: Laconic function evaluation and ABE for RAMs from (ring-)LWE. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part III. LNCS, vol. 14922, pp. 107–142. Springer, Cham (Aug 2024)
26. Dong, F., Hao, Z., Mook, E., Wichs, D.: Laconic function evaluation, functional encryption and obfuscation for RAMs with sublinear computation. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part II. LNCS, vol. 14652, pp. 190–218. Springer, Cham (May 2024)
27. Döttling, N., Kolonelos, D., Lai, R.W.F., Lin, C., Malavolta, G., Rahimi, A.: Efficient laconic cryptography from learning with errors. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part III. LNCS, vol. 14006, pp. 417–446. Springer, Cham (Apr 2023)
28. Dujmovic, J., Hajiabadi, M.: Lower-bounds on public-key operations in PIR. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part VI. LNCS, vol. 14656, pp. 65–87. Springer, Cham (May 2024)
29. Fleischhacker, N., Hall-Andersen, M., Simkin, M.: Extractable witness encryption for kzg commitments and efficient laconic ot. In: ASIACRYPT. Springer-Verlag (2024)
30. Garg, S., Hajiabadi, M., Mahmoody, M., Mohammed, A.: Limits on the power of garbling techniques for public-key encryption. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part III. LNCS, vol. 10993, pp. 335–364. Springer, Cham (Aug 2018)
31. Garg, S., Lu, S., Ostrovsky, R.: Black-box garbled RAM. In: Guruswami, V. (ed.) 56th FOCS. pp. 210–229. IEEE Computer Society Press (Oct 2015)
32. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: STOC. pp. 169–178 (2009)
33. Gentry, C., Halevi, S., Lu, S., Ostrovsky, R., Raykova, M., Wichs, D.: Garbled RAM revisited. In: Nguyen, P.Q., Oswald, E. (eds.) EUROCRYPT 2014. LNCS, vol. 8441, pp. 405–422. Springer, Berlin, Heidelberg (May 2014)
34. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: CRYPTO. pp. 75–92. Springer (2013)
35. Ghosh, S., Nielsen, J.B., Nilges, T.: Maliciously secure oblivious linear function evaluation with constant overhead. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part I. LNCS, vol. 10624, pp. 629–659. Springer, Cham (Dec 2017)

36. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions (extended abstract). In: 25th FOCS. pp. 464–479. IEEE Computer Society Press (Oct 1984)
37. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game or A completeness theorem for protocols with honest majority. In: Aho, A. (ed.) 19th ACM STOC. pp. 218–229. ACM Press (May 1987)
38. Green, M., Jain, A., Laer, G.V.: Efficient set membership encryption and applications. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 1080–1092. ACM Press (Nov 2023)
39. Hanaoka, G., Imai, H., Müller-Quade, J., Nascimento, A.C.A., Otsuka, A., Winter, A.: Information theoretically secure oblivious polynomial evaluation: Model, bounds, and constructions. In: Wang, H., Pieprzyk, J., Varadharajan, V. (eds.) ACISP 04. LNCS, vol. 3108, pp. 62–73. Springer, Berlin, Heidelberg (Jul 2004)
40. Hazay, C., Orsini, E., Scholl, P., Soria-Vazquez, E.: Concretely efficient large-scale MPC with active security (or, TinyKeys for TinyOT). In: Peyrin, T., Galbraith, S. (eds.) ASIACRYPT 2018, Part III. LNCS, vol. 11274, pp. 86–117. Springer, Cham (Dec 2018)
41. Heath, D., Kolesnikov, V., Ostrovsky, R.: EpiGRAM: Practical garbled RAM. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part I. LNCS, vol. 13275, pp. 3–33. Springer, Cham (May / Jun 2022)
42. Impagliazzo, R., Levin, L.A., Luby, M.: Pseudo-random generation from one-way functions (extended abstracts). In: 21st ACM STOC. pp. 12–24. ACM Press (May 1989)
43. Impagliazzo, R., Rudich, S.: Limits on the provable consequences of one-way permutations. In: Goldwasser, S. (ed.) CRYPTO’88. LNCS, vol. 403, pp. 8–26. Springer, New York (Aug 1990)
44. Ishai, Y., Kilian, J., Nissim, K., Petrank, E.: Extending oblivious transfers efficiently. In: Boneh, D. (ed.) CRYPTO 2003. LNCS, vol. 2729, pp. 145–161. Springer, Berlin, Heidelberg (Aug 2003)
45. Ishai, Y., Prabhakaran, M., Sahai, A.: Secure arithmetic computation with no honest majority. In: Reingold, O. (ed.) TCC 2009. LNCS, vol. 5444, pp. 294–314. Springer, Berlin, Heidelberg (Mar 2009)
46. Kilian, J.: Founding cryptography on oblivious transfer. In: 20th ACM STOC. pp. 20–31. ACM Press (May 1988)
47. Lu, S., Ostrovsky, R.: How to garble RAM programs. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 719–734. Springer, Berlin, Heidelberg (May 2013)
48. Naor, M., Pinkas, B.: Oblivious polynomial evaluation. *SIAM J. Comput.* 35(5), 1254–1281 (2006)
49. Nielsen, J.B., Nordholt, P.S., Orlandi, C., Burra, S.S.: A new approach to practical active-secure two-party computation. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 681–700. Springer, Berlin, Heidelberg (Aug 2012)
50. Quach, W., Wee, H., Wichs, D.: Laconic function evaluation and applications. In: Thorup, M. (ed.) 59th FOCS. pp. 859–870. IEEE Computer Society Press (Oct 2018)
51. Rabin, M.O.: How to exchange secrets with oblivious transfer. Technical Report TR-81, Aiken Computation Lab, Harvard University (1981)
52. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC. pp. 84–93. ACM Press (May 2005)

- 53. Yang, K., Weng, C., Lan, X., Zhang, J., Wang, X.: Ferret: Fast extension for correlated OT with small communication. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020. pp. 1607–1626. ACM Press (Nov 2020)
- 54. Yao, A.C.C.: Protocols for secure computations (extended abstract). In: 23rd FOCS. pp. 160–164. IEEE Computer Society Press (Nov 1982)